DD: WO-01-FRAMEWORK — Work Order Module Framework

Developer implementation guide: `DD_WO-01-FRAMEWORK-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the module contract `DD_WO-01-FRAMEWORK-v1.0.md`.

Verdict: ■ New | **Wave:** 2 | **Group III:** Fulfillment & Operations | **Version:** v1.0 | **Module:** Work Order The framework satellite for the Work Order module. Defines the generic primitives every Work Order flow builds on: the state machine, REST API surface, structured-notes mechanism, attachments, contractor + team assignment, skills filtering, master/sub linkage, multi-phase BPMN support, finalization checklist enforcement, SLA timestamp capture, audit trail, Kafka event catalog. Does NOT define flows — Installation, Support, Shifting flows live in their own satellites (Phase B). The framework is what those flows reuse. Per memory rule #11, this satellite is self-contained. A developer implementing a WO flow needs only this framework + their flow satellite.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/07_work_order/DD_WO-01-FRAMEWORK-v1.0.md`
Version	`v1.0`
DD type	`module contract`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `GET /api/customers/{id}`
- `GET /api/work-orders`
- `GET /api/work-orders/wo_01HZ...`
- `GET /api/work-orders/wo_01HZ.../print-pdf`
- `GET /api/work-orders/{woId}`

- GET /api/work-orders/{woId}/print-pdf
- GET /api/work-orders?operatorCode=WIK&status=PENDING,ASSIGNED&kind=SUPPORT&techRegionCode=NRB-KAREN&limit=50
- GET /engine-rest/process-instance/{id}/variables/{var}
- POST /api/work-orders
- POST /api/work-orders/wo_01HZ.../assign
- POST /api/work-orders/wo_01HZ.../attachments
- POST /api/work-orders/wo_01HZ.../cancel
- POST /api/work-orders/wo_01HZ.../claim
- POST /api/work-orders/wo_01HZ.../finalize-first-confirm
- POST /api/work-orders/wo_01HZ.../finalize-revert
- POST /api/work-orders/wo_01HZ.../notes
- POST /api/work-orders/wo_01HZ.../start
- POST /api/work-orders/wo_01HZ.../transition-phase
- POST /api/work-orders/wo_01HZ...T2/assign
- POST /api/work-orders/wo_01HZ...T2/claim
- POST /api/work-orders/wo_01HZ...T3/assign
- POST /api/work-orders/wo_01HZ...T3/claim
- POST /api/work-orders/wo_01HZ...T3/start
- POST /api/work-orders/wo_01HZ...Z2/assign
- POST /api/work-orders/wo_01HZ...Z2/start
- POST /api/work-orders/{id}/assign
- POST /api/work-orders/{id}/cancel
- POST /api/work-orders/{id}/claim
- POST /api/work-orders/{id}/equipment-bindings
- POST /api/work-orders/{id}/finalize-first-confirm
- POST /api/work-orders/{id}/finalize-revert
- POST /api/work-orders/{id}/finalize-second-confirm
- POST /api/work-orders/{id}/reassign
- POST /api/work-orders/{id}/start
- POST /api/work-orders/{id}/transition-phase
- POST /api/work-orders/{woId}/assign
- POST /api/work-orders/{woId}/attachments
- POST /api/work-orders/{woId}/cancel
- POST /api/work-orders/{woId}/claim
- POST /api/work-orders/{woId}/finalize-first-confirm
- POST /api/work-orders/{woId}/finalize-revert
- POST /api/work-orders/{woId}/finalize-second-confirm
- POST /api/work-orders/{woId}/notes
- POST /api/work-orders/{woId}/reassign
- POST /api/work-orders/{woId}/start
- POST /api/work-orders/{woId}/transition-phase

- `POST /engine-rest/deployment/create`
- `POST /engine-rest/external-task/fetchAndLock`
- `POST /engine-rest/message`
- `POST /engine-rest/process-definition/key/WorkOrder_Installation_WIK/start`
- `POST /start`

4.2 Tables

- `wo_equipment_ref_stub`

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `await-contractor-dispatcher-pick`
- `await-finalize-second-confirm`
- `await-tech-start`
- `fulfillment-order-capture-worker-7`
- `order-create-install-wo`
- `wait-install-wo-finalized`
- `wo-auto-assign-contractor`
- `wo-emit-state-event`
- `wo-enforce-finalize-checklist`
- `wo-mark-cancelled`
- `wo-shifting-update-address-on-finalize`
- `wo-validate-create`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- `WorkOrderAssigned`
- `WorkOrderAttachmentAdded`
- `WorkOrderCancellationRequested`
- `WorkOrderCancelled`
- `WorkOrderClaimedByDispatcher`
- `WorkOrderClaimedByTechnician`
- `WorkOrderCreated`
- `WorkOrderDispatched`

- `WorkOrderEscalationCandidate`
- `WorkOrderFinalizationPending`
- `WorkOrderFinalized`
- `WorkOrderInProgress`
- `WorkOrderNoteAppended`
- `WorkOrderPhaseTransitioned`
- `WorkOrderReassigned`
- `WorkOrderUserActionReceived`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group III:** Fulfillment & Operations | **Version:** v1.0 | **Module:** Work Order

The framework satellite for the Work Order module. Defines the generic primitives every Work Order flow builds on: the state machine, REST API surface, structured-notes mechanism, attachments, contractor + team assignment, skills filtering,

master/sub linkage, multi-phase BPMN support, finalization checklist enforcement, SLA timestamp capture, audit trail, Kafka event catalog. Does NOT define flows — Installation, Support, Shifting flows live in their own satellites (Phase B). The framework is what those flows reuse.

Per memory rule #11, this satellite is self-contained. A developer implementing a WO flow needs only this framework + their flow satellite.

1. What the framework is

The WO module owns one thing: **coordinating units of work between teams**. A Work Order is the durable record of "this needs doing, here's who's doing it, here's what happened along the way, here's how it ended." It does not own the *business meaning* of the work — that lives with the consumer (FUL-02 creates install WOs as part of the customer-onboarding journey; the future Ticketing module will create support WOs from customer tickets; OSR-* will create WOs for MACD operations on existing subscriptions). The WO module owns the *coordination machinery* those consumers need.

The framework provides:

1. A generic state machine (PENDING → ASSIGNED → IN_PROGRESS → FINALIZATION_PENDING → COMPLETED / CANCELLED) that every flow uses unchanged.
2. A REST API surface for creating, querying, assigning, noting, attaching to, finalizing, and cancelling Work Orders — the same API serves FUL-02, Ticketing, OSR-*, and the dispatcher BO UI.
3. A structured-notes mechanism — operators register `note_kind` schemas; flows declare which kinds are required before a WO can finalize; the framework enforces.
4. Attachments with category + description + file URI.
5. Multi-phase BPMN support — a WO can be in `status=IN_PROGRESS` while progressing through `current_phase=DISCONNECT` then `current_phase=RECONNECT` within one process instance. The Shifting flow uses this; future flows can too.
6. Master/sub linkage primitive — a WO can carry `master_wo_id` and `link_type` (`ESCALATION`, `REPEAT_WITHIN_WARRANTY`, `PAIRED`). v1.0 ships the primitive; v1.0 flows don't actively use it (Ticketing will, per Phase B).
7. Contractor + team assignment model with skills filtering and schedule availability.
8. 2-step finalization confirm — first confirm saves modifications + transitions to FINALIZATION_PENDING; second confirm executes finalization checklist + closes.
9. SLA timestamp capture — every phase transition timestamped. Calc deferred to Phase 2 / future FUL-OPS-SLA DD.
10. Audit trail — every state change, every action (note appended, attachment added, reassignment, print) recorded with actor and timestamp.
11. Kafka events for state transitions consumed by downstream modules (FUL-02 listens on `WorkOrderFinalized` + `WorkOrderCancelled` today).
12. Camunda integration — one BPMN per flow (Installation, Support, Shifting, future flows). Workers are framework-provided primitives; flows wire them up.

A Camunda BPMN is a flowchart-like XML file describing the sequence of steps an operation goes through; "process instance" is one running execution. A "worker" is a Java bean inside our module that long-polls Camunda for work on a named topic, does the work, reports back; an "external task" is the BPMN step where Camunda hands work to a worker. A "user task" is a BPMN step waiting for a human; an "HTTP connector" fires when a user task is created and pings our notification handler so the responsible team is told there's work waiting.

What the framework does NOT do

- **Doesn't interpret the work's meaning.** "Install a GPON modem" vs "investigate a TV signal complaint" are equally opaque to the framework — both are WOs with consumer-defined kind + structured notes.
- **Doesn't integrate with external tools** (Easydocsis, Google Maps, mobile field app, WhatsApp). Field tech reads optical levels from Easydocsis manually and submits them as a structured note via the WO REST API or BO UI form. The framework receives the JSONB payload and validates it against the registered schema — but the *content* of the readings is operator-defined.

- **Doesn't compute SLA.** Timestamps are captured; calc lives in future FUL-OPS-SLA module (per GAP roadmap Phase 2).
- **Doesn't escalate WOs autonomously.** The Pattern A escalation (Support WO → QCS WO to NOC) is owned by the future Ticketing module: WO emits `WorkOrderFinalized` with the final reason, Ticketing decides whether and how to escalate (creating a new WO via the same REST API). The framework provides the primitives — the policy belongs elsewhere.
- **Doesn't own the inventory of equipment + parts** assigned to a WO. The WO references equipment by ID; the contractor-stock + per-tech inventory layer is the future Inventory module.
- **Doesn't replace channels** like WhatsApp. v1.0 framework supports structured notes via REST API; how the notes get *into* the API (BO UI form, future mobile app, manual paste) is a channel concern.

Glossary

Term	Meaning
WO	Work Order — one unit of coordinated work tracked by this module
kind	High-level discriminator set by the consumer (`INSTALLATION`, `SUPPORT`, `SHIFTING`). Drives BPMN selection.
job_type	Fine-grained code from operator catalog (`GP3`, `HSD`, `EQU`). Drives Drools rules, note-kind requirements, picklist filtering.
status	Generic state-machine value: `PENDING`, `ASSIGNED`, `IN_PROGRESS`, `FINALIZATION_PENDING`, `COMPLETED`, `CANCELLED`
current_phase	Flow-defined sub-state within IN_PROGRESS. Shifting uses `DISCONNECT` → `RECONNECT`. Optional.
note	Append-only typed entry on a WO. Has a `note_kind` (free-text or structured JSONB schema registered per operator).
attachment	File + category + description on a WO.
slot	Contractor-named container (e.g., `Z071H In-House Support`) where WOs assigned to a contractor land.
tech_region	Geographic / operational region a technician belongs to. Filters which WOs they can receive.
skill	Service Plan Type (`INT` / `OTH` / `PHN` / `VID`) + Technician Work Type (per job_type code).
master/sub linkage	A WO can have `master_wo_id` pointing to another WO with `link_type` discriminator.
flow	A BPMN file + Drools KJAR + operator config combination defining one type of WO journey.
2-step finalize	First confirm transitions IN_PROGRESS → FINALIZATION_PENDING (saves notes + attachments + final reason); second confirm runs the required-checklist validation and transitions to COMPLETED.

2. Camunda primitives

The framework uses the same Camunda integration pattern as FUL-02 (see DD_FUL-02-FRAMEWORK-v1.0.md §1). Repeating the essentials here so this satellite is self-contained.

2.1 Start a process instance

When a WO is created via `POST /api/work-orders`, the module:

1. Inserts the row in `work_order` table (`status = PENDING`).
2. Starts a Camunda process instance for the appropriate BPMN (`WorkOrder_Installation`, `WorkOrder_Support`, `WorkOrder_Shifting` — derived from `kind`). Uses `businessKey = wo_id`.
3. Updates the row with the returned `process_instance_id`.

```
POST /engine-rest/process-definition/key/WorkOrder_Installation_WIK/start HTTP/1.1
Host: camunda.sophix.internal
Content-Type: application/json

{
  "businessKey": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "variables": {
```

```
    "operatorCode":     { "value": "WIK",               "type": "String" },
    "woId":             { "value": "wo_01HZ...",        "type": "String" },
    "kind":             { "value": "INSTALLATION",      "type": "String" },
    "jobType":          { "value": "H01",               "type": "String" },
    "customerId":       { "value": "cust_01HX...",      "type": "String" },
    "homepassId":       { "value": "hp_01HX...",        "type": "String" },
    "originatingContextType": { "value": "ORDER",       "type": "String" },
    "originatingContextRef":  { "value": "ord_01HZ...", "type": "String" }
  }
}
```

Compensating transaction: if Camunda start fails after the row was written, the row is DELETEd. Caller retries with the same `idempotency_key`.

2.2 Worker long-poll

Workers fetch their tasks via `POST /engine-rest/external-task/fetchAndLock`. The framework defines these worker topics (all v1.0):

```
{
  "workerId": "work-order-worker-1",
  "asyncResponseTimeout": 30000,
  "maxTasks": 10,
  "topics": [
    { "topicName": "wo-validate-create",             "lockDuration": 5000 },
    { "topicName": "wo-auto-assign-contractor",      "lockDuration": 10000 },
    { "topicName": "wo-evaluate-skills-match",       "lockDuration": 5000 },
    { "topicName": "wo-enforce-finalize-checklist",  "lockDuration": 5000 },
    { "topicName": "wo-emit-state-event",            "lockDuration": 3000 },
    { "topicName": "wo-render-print-pdf",            "lockDuration": 15000 },
    { "topicName": "wo-trigger-side-effect-event",   "lockDuration": 5000 },
    { "topicName": "wo-mark-cancelled",              "lockDuration": 5000 }
  ]
}
```

Flow-specific worker topics (e.g., `wo-shifting-update-address-on-finalize`) live in the flow satellites and add to this list. Framework workers are reusable; flow workers are flow-owned.

2.3 Message catch events

Two framework-level message catches any flow can use:

Message	What wakes it	When it fires
`WorkOrderUserActionReceived`	The dispatcher / contractor performs an action via REST API that the BPMN was waiting for	The corresponding REST endpoint calls `POST /engine-rest/message`
`WorkOrderCancellationRequested`	Dispatcher cancels the WO via `POST /api/work-orders/{id}/cancel`	Same — REST endpoint sends the Camunda message

Flows add their own message catches for flow-specific external signals (see flow satellites).

2.4 BPMN snippet — Cancellation as interrupting boundary

Every flow BPMN attaches a top-level interrupting boundary message event on `WorkOrderCancellationRequested` that interrupts whatever subprocess is running and routes to `wo-mark-cancelled`. This is the framework-provided cancellation pattern; flows inherit it.

```
<bpmn:subProcess id="wo-main-subprocess">
  <!-- flow-specific sequence -->
</bpmn:subProcess>

<bpmn:boundaryEvent id="boundary-cancel"
                    attachedToRef="wo-main-subprocess"
                    cancelActivity="true">
  <bpmn:messageEventDefinition messageRef="msg-wo-cancellation-requested"/>
</bpmn:boundaryEvent>

<bpmn:sequenceFlow sourceRef="boundary-cancel" targetRef="wo-cancel-cascade"/>
```

2.5 BPMN snippet — User Task with HTTP connector

When a BPMN needs human action (a dispatcher must assign, a contractor must confirm), the framework uses Camunda user tasks with HTTP connectors firing notification to the WO module's `NotificationHandlerResource` — same pattern as FUL-02 FRAMEWORK §1.6. The handler dispatches to email / Slack / Teams via `staff_notification_stub` (v1.0) or ICN-01 (future).

2.6 Variable read API

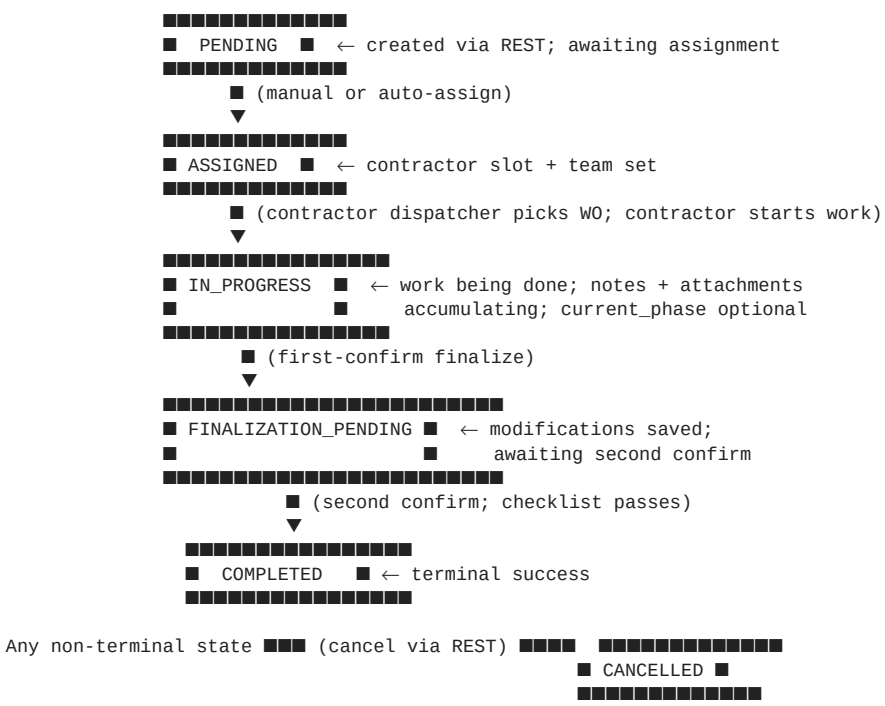
Workers can read Camunda process variables via `GET /engine-rest/process-instance/{id}/variables/{var}`. Used by reassignment workers reading the previous assignment state.

2.7 Deployment lifecycle

Flow BPMNs are deployed via `POST /engine-rest/deployment/create`. Each flow's deploy step is documented in the flow satellite.

3. State machine

The framework defines one generic state machine. Every flow uses it unchanged; flows add phase sub-states via `current_phase` if they need multi-step IN_PROGRESS.



Status transitions:

From	To	Trigger
(none)	PENDING	`POST /api/work-orders`
PENDING	ASSIGNED	`POST /api/work-orders/{id}/assign` with contractor + team (manual or auto-assignment worker output)
PENDING	CANCELLED	`POST /api/work-orders/{id}/cancel`
ASSIGNED	IN_PROGRESS	`POST /api/work-orders/{id}/start` (contractor confirms on-site / desk work begun)
ASSIGNED	CANCELLED	`POST /api/work-orders/{id}/cancel`
ASSIGNED	ASSIGNED	`POST /api/work-orders/{id}/reassign` — new contractor + team; logged in `wo_assignment_history`; status doesn't change
IN_PROGRESS	IN_PROGRESS	`POST /api/work-orders/{id}/reassign` — Pattern B (installation passed to maintenance without closure); logged; status doesn't change
IN_PROGRESS	IN_PROGRESS	`POST /api/work-orders/{id}/transition-phase` — multi-phase flows like Shifting (`current_phase`: DISCONNECT → RECONNECT)
IN_PROGRESS	FINALIZATION_PENDING	`POST /api/work-orders/{id}/finalize-first-confirm`
IN_PROGRESS	CANCELLED	`POST /api/work-orders/{id}/cancel`

From	To	Trigger
FINALIZATION_PENDING	IN_PROGRESS	`POST /api/work-orders/{id}/finalize-revert` (dispatcher catches a mistake)
FINALIZATION_PENDING	COMPLETED	`POST /api/work-orders/{id}/finalize-second-confirm` — checklist must pass
FINALIZATION_PENDING	CANCELLED	`POST /api/work-orders/{id}/cancel`

Every transition emits a Kafka event (§7) and writes a row to `wo_status_history`. The status column on `work_order` always reflects the latest state; the history table is the audit log.

4. Per-operator + per-flow configuration

The framework reads operator-scoped + flow-scoped configuration to enforce business rules. v1.0 seeds WIK; the schema supports WUG, WTZ, YASSN with no DDL changes.

4.1 `wo_flow_config`

Per `(operator_code, kind)` row defining which BPMN to start and which Drools KJAR to use.

Column	Type	Notes
`operator_code`	TEXT NOT NULL	`WIK` / `WUG` / `WTZ` / `YASSN`
`kind`	TEXT NOT NULL	`INSTALLATION` / `SUPPORT` / `SHIFTING` — future kinds added without DDL
`bpmn_process_key`	TEXT NOT NULL	Camunda process key. Example: `WorkOrder_Installation_WIK`
`drools_kjar_ref`	TEXT NULL	Drools KJAR for flow decisions. Example: `wo-installation-wik-v1.0.kjar`
`created_at`	TIMESTAMP NOT NULL	

PK `(operator_code, kind)`.

Sample data (v1.0 KE seed):

operator_code	kind	bpmn_process_key	drools_kjar_ref
`WIK`	`INSTALLATION`	`WorkOrder_Installation_WIK`	`wo-installation-wik-v1.0.kjar`
`WIK`	`SUPPORT`	`WorkOrder_Support_WIK`	`wo-support-wik-v1.0.kjar`
`WIK`	`SHIFTING`	`WorkOrder_Shifting_WIK`	`wo-shifting-wik-v1.0.kjar`

4.2 `wo_job_type_catalog`

Per-operator catalog of job types. The KE catalog (workshop 2026-04-29) has ~30 codes; YASSN, WUG, WTZ will seed their own catalogs when their flows ship.

Column	Type	Notes
`operator_code`	TEXT NOT NULL	
`job_type_code`	TEXT NOT NULL	Example: `H01`, `GP3`, `GSD`, `QCS`, `EQU`
`kind`	TEXT NOT NULL	Which flow this job type belongs to (`INSTALLATION` / `SUPPORT` / `SHIFTING`)
`display_name`	TEXT NOT NULL	Example: `GPON Internet Service Call`
`description`	TEXT NULL	Example: `LOS, drop cut, internet issues for GPON`
`network_type`	TEXT NULL	`GPON` / `HFC` / NULL for technology-agnostic
`requires_site_visit`	BOOLEAN NOT NULL	`false` for GSD/HSD (system-driven); `true` for most others
`warranty_days`	INTEGER NOT NULL DEFAULT 0	Example: `90` (Support), `180` (Installation), `0` (no warranty)
`active`	BOOLEAN NOT NULL DEFAULT true	

Column	Type	Notes
created_at	TIMESTAMP NOT NULL	

PK (operator_code, job_type_code). Index (operator_code, kind, active).

Sample data (v1.0 WIK seed, partial — full catalog in seed migration):

operator_code	job_type_code	kind	display_name	network_type	requires_site_visit	warranty_days
WIK	H01	INSTALLATION	HFC Installation	HFC	true	180
WIK	G07	INSTALLATION	VOIP Installation	NULL	true	180
WIK	GP3	SUPPORT	GPON Internet Service Call	GPON	true	90
WIK	HS1	SUPPORT	HFC Internet Support	HFC	true	90
WIK	GSD	SHIFTING	GPON Shifting Disconnect	GPON	false	0
WIK	GSR	SHIFTING	GPON Shifting Reconnect	GPON	true	0
WIK	HSD	SHIFTING	HFC Shifting Disconnect	HFC	false	0
WIK	HSR	SHIFTING	HFC Shifting Reconnect	HFC	true	0
WIK	QCS	SUPPORT	Quality Control Service (Escalation)	NULL	false	0
WIK	EQU	INSTALLATION	Equipment Upgrade	NULL	true	0
WIK	RPT	SUPPORT	Repeat Work Order	NULL	true	0

4.3 wo_note_kind_registry

Per-operator catalog of note kinds. The framework reads this at note-write time (validates the JSONB payload) and at finalization time (checks required notes present).

Column	Type	Notes
operator_code	TEXT NOT NULL	
note_kind	TEXT NOT NULL	Example: findings, solution, optical_readings, hfc_signal_readings, csr_note, dispatcher_note, customer_appointment
schema_jsonb	JSONB NULL	JSON Schema. NULL = free-text note (no validation). Example for optical_readings: {"type":"object","required":["oltPort","onuId","distanceM","opticalTempC","ontRxDbm","ontTxDbm","oltRxDbm"]}
append_only	BOOLEAN NOT NULL DEFAULT true	Append-only is the default; some kinds (e.g., customer_appointment) may allow update
created_at	TIMESTAMP NOT NULL	

PK (operator_code, note_kind).

Sample data (v1.0 WIK seed, partial):

operator_code	note_kind	schema_jsonb (excerpt)	append_only
WIK	findings	NULL (free text)	true
WIK	solution	NULL (free text)	true
WIK	optical_readings	{"required":["oltPort","onuId","ontRxDbm","ontTxDbm","oltRxDbm","distanceM"]}	true
WIK	hfc_signal_readings	{"required":["downstreamLevel","upstreamLevel","snr"]}	true

operator_code	note_kind	schema_jsonb (excerpt)	append_only
WIK	csr_note	NULL (free text)	true
WIK	dispatcher_note	NULL (free text)	true
WIK	customer_appointment	{"required":["scheduledAt"]}	false
WIK	whatsapp_paste	NULL (free text — v1.0 fallback for field tech reports pasted from WhatsApp)	true
WIK	final_reason_set	{"required":["finalReasonCode"]}	true
WIK	new_address	{"required":["addressFreeText"],"properties":{"homepassId":{"type":"string"}}}	false

4.4 wo_finalization_requirements

Per (operator_code, kind, job_type) row stating what must be present before second-confirm finalization succeeds. The framework's wo-enforce-finalize-checklist worker reads this at second-confirm and rejects if anything's missing.

Column	Type	Notes
operator_code	TEXT NOT NULL	
kind	TEXT NOT NULL	INSTALLATION / SUPPORT / SHIFTING
job_type_code	TEXT NULL	NULL = applies to all job types of this kind for this operator (default rule); specific job type code overrides the default
required_note_kinds	JSONB NOT NULL	Array. Example: ["findings","solution","optical_readings","final_reason_set"]
required_attachment_categories	JSONB NOT NULL	Array. Example: ["setup_photo","rf_optical_reading","speed_test"]
min_attachments_per_category	JSONB NULL	Per-category minimum count. Example: {"setup_photo":1,"rf_optical_reading":1}. NULL = 1 of each required category
created_at	TIMESTAMP NOT NULL	

PK (operator_code, kind, COALESCE(job_type_code, '*')).

Sample data (v1.0 WIK seed):

operator_code	kind	job_type_code	required_note_kinds	required_attachment_categories
WIK	INSTALLATION	NULL	["findings","solution","final_reason_set"]	["setup_photo"]
WIK	INSTALLATION	H01	["findings","solution","hfc_signal_readings","final_reason_set"]	["setup_photo","speed_test"]
WIK	INSTALLATION	G07	["findings","solution","final_reason_set"]	["setup_photo"]
WIK	SUPPORT	NULL	["findings","solution","final_reason_set"]	[]
WIK	SUPPORT	GP3	["findings","solution","optical_readings","final_reason_set"]	["rf_optical_reading"]
WIK	SUPPORT	HS1	["findings","solution","hfc_signal_readings","final_reason_set"]	["rf_optical_reading","speed_test"]
WIK	SHIFTING	GSD	["dispatcher_note"]	[]
WIK	SHIFTING	GSR	["findings","solution","optical_readings","new_address","final_reason_set"]	["setup_photo"]

Validation logic at second-confirm finalize: for the WO's (operator_code, kind, job_type), read the row (specific job_type overriding the default job_type_code=NULL row); verify every required_note_kind has ≥1 entry in

wo_notes; verify every required_attachment_category has ≥ the minimum count in wo_attachments. On miss → return 400 FINALIZATION_CHECKLIST_FAILED with the list of missing items.

4.5 wo_final_reason_catalog

Per (operator_code, kind, job_type) picklist of valid final reasons.

Column	Type	Notes
operator_code	TEXT NOT NULL	
kind	TEXT NOT NULL	
job_type_code	TEXT NULL	NULL = applies to all job types of this kind
final_reason_code	TEXT NOT NULL	Example: RESOLVED_ON_SITE, NO_FAULT_FOUND, CABLE_REPLACED, COULD_NOT_RESOLVE_ESCALATED
display_name	TEXT NOT NULL	Example: Resolved on site
triggers_escalation_event	BOOLEAN NOT NULL DEFAULT false	If true, finalization emits an additional Kafka event WorkOrderEscalationCandidate consumed by Ticketing (future)
active	BOOLEAN NOT NULL DEFAULT true	

PK (operator_code, kind, COALESCE(job_type_code, '*'), final_reason_code).

Sample data (v1.0 WIK seed, partial):

operator_code	kind	job_type_code	final_reason_code	display_name	triggers_escalation_event
WIK	INSTALLATION	NULL	INSTALL_COMPLETED_CUSTOMER_UP	Install completed, customer up	false
WIK	INSTALLATION	NULL	CUSTOMER_NOT_AVAILABLE	Customer not available	false
WIK	SUPPORT	NULL	RESOLVED_ON_SITE	Resolved on site	false
WIK	SUPPORT	NULL	NO_FAULT_FOUND	No fault found	false
WIK	SUPPORT	NULL	CABLE_REPLACED	Faulty cable replaced	false
WIK	SUPPORT	NULL	COULD_NOT_RESOLVE_ESCALATED	Could not resolve — escalate to NOC	**true**
WIK	SUPPORT	NULL	AREA_OUTAGE_OPEN	Area outage — outage handled separately	false
WIK	SHIFTING	GSD	DISCONNECTED_SUCCESSFULLY	Disconnected successfully	false
WIK	SHIFTING	GSR	RECONNECTED_AT_NEW_ADDRESS	Reconnected at new address	false

5. Core schemas

5.1 work_order

The main table. One row per WO. All Camunda state changes project onto this row's status + current_phase + *_at timestamp columns via the wo-emit-state-event worker.

Column	Type	Notes
wo_id	TEXT PK	ULID. Example: wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
operator_code	TEXT NOT NULL	WIK
kind	TEXT NOT NULL	INSTALLATION / SUPPORT / SHIFTING
job_type_code	TEXT NOT NULL	From wo_job_type_catalog. Example: H01
network_type	TEXT NULL	Denormalized from wo_job_type_catalog. GPON / HFC / NULL
customer_id	TEXT NULL	FK to ILM-CFG-01 (v1.0 stub customer_stub). Example: cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6

Column	Type	Notes
homepass_id	TEXT NULL	FK to RLM-CFG-01. Example: hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P
originating_context_type	TEXT NULL	ORDER (FUL-02), TICKET (future Ticketing), MASTER_WO (escalation/repeat), OSR (future OSR-*), BACKOFFICE (manual creation). Extensible.
originating_context_ref	TEXT NULL	FK-shaped reference. Example: ord_01HZ...
idempotency_key	TEXT NOT NULL	Consumer-supplied dedup key
status	TEXT NOT NULL	Generic state. Values from §3. Example: IN_PROGRESS
current_phase	TEXT NULL	Flow-defined sub-state within IN_PROGRESS. NULL for flows that don't use phases. Example: DISCONNECT (Shifting flow)
assigned_contractor_id	TEXT NULL	FK to contractor registry. Example: ctr_Z071H
assigned_team_id	TEXT NULL	FK to team registry. Example: team_rapid_response
assigned_technician_id	TEXT NULL	FK to technician registry. In TECHNICIAN mode: set at assignment. In TEAM mode (internal team): NULL until a team member claims via /claim. In TEAM mode (external team): NULL permanently. Example: tech_01HZ_KAREN_03
assignment_mode	TEXT NULL	TECHNICIAN (specific tech picked at assignment) or TEAM (team-level; pending claim or external-dispatcher ack). NULL when WO is in PENDING (not yet assigned).
claimed_at	TIMESTAMP NULL	When the WO was claimed. In TECHNICIAN mode equals assigned_at; in TEAM mode set by /claim (either real tech or external dispatcher).
claimed_by_dispatcher	TEXT NULL	BO user ID who acknowledged a TEAM-mode WO for an external contractor (the contractor's dispatcher or the Wananchi-side staff member coordinating with the external contractor). NULL when an internal technician claimed via real-tech /claim.
tech_region_code	TEXT NULL	From HomePass; drives contractor routing. Example: NRB-KAREN
scheduled_at	TIMESTAMP NULL	Time slot for the visit. Set when dispatcher schedules.
master_wo_id	TEXT NULL	FK to another work_order.wo_id. v1.0 primitive ready for Ticketing escalation + RPT linkage.
link_type	TEXT NULL	When master_wo_id is set: ESCALATION, REPEAT_WITHIN_WARRANTY, PAIRED. Extensible.
final_reason_code	TEXT NULL	Set at finalization. From wo_final_reason_catalog.
cancellation_reason	TEXT NULL	Set at cancellation. Operator-defined enum. Example: CUSTOMER_DECLINED
cancellation_detail	TEXT NULL	Free text.
process_instance_id	TEXT NULL	Camunda process instance ID.
process_definition_key	TEXT NOT NULL	Camunda process key. Example: WorkOrder_Installation_WIK
created_at	TIMESTAMP NOT NULL DEFAULT NOW()	
created_by	TEXT NOT NULL	Caller ID. Example: fulfillment-order-capture-worker-7 (FUL-02); csr_user_42 (BO UI)
assigned_at	TIMESTAMP NULL	First time status reached ASSIGNED.
dispatched_at	TIMESTAMP NULL	When contractor's dispatcher assigned to a specific tech.
in_progress_at	TIMESTAMP NULL	When status reached IN_PROGRESS.
finalization_pending_at	TIMESTAMP NULL	First-confirm finalize timestamp.
completed_at	TIMESTAMP NULL	Second-confirm finalize timestamp.
cancelled_at	TIMESTAMP NULL	Cancellation timestamp.

Column	Type	Notes
last_updated_at	TIMESTAMP NOT NULL DEFAULT NOW()	Updated on every UPDATE.
metadata	JSONB NULL	Flow-defined extension payload. Example for Shifting RECONNECT: {"oldHomepassId":"hp_01HX...","newAddressFreeText":"Plot 47, Westlands"}

Indexes: PK; UNIQUE (operator_code, originating_context_type, idempotency_key); (operator_code, status, created_at DESC); (operator_code, kind, status); (assigned_contractor_id, status); (assigned_technician_id, scheduled_at); (customer_id, created_at DESC); (homepass_id); partial (master_wo_id) WHERE master_wo_id IS NOT NULL; partial (process_instance_id) WHERE process_instance_id IS NOT NULL. P10Y retention.

Sample data:

wo_id	operator	kind	job_type	status	mode	contractor	tech	claimed_by_dispatcher
wo_01HZ...A	WIK	INSTALLATION	H01	COMPLETED	TECHNICIAN	ctr_Z071H (internal)	tech_01HZ_KAREN_03	NULL
wo_01HZ...B	WIK	INSTALLATION	H01	IN_PROGRESS	TECHNICIAN	ctr_Z071H (internal)	tech_01HZ_KAREN_05	NULL
wo_01HZ...C	WIK	SUPPORT	GP3	IN_PROGRESS	TEAM	ctr_GTECH (external)	NULL	dispatcher_user_gtech_01 (ack-only)
wo_01HZ...D	WIK	SHIFTING	GSD	IN_PROGRESS	TECHNICIAN	ctr_Z071H (internal)	tech_01HZ_KAREN_07	NULL
wo_01HZ...E	WIK	SHIFTING	GSR	IN_PROGRESS	TECHNICIAN	ctr_Z071H (internal)	tech_01HZ_KILELE_02	NULL
wo_01HZ...F	WIK	SUPPORT	QCS	IN_PROGRESS	TECHNICIAN	ctr_NOC_MAINT (internal)	noc_tech_03	NULL
wo_01HZ...G	WIK	SUPPORT	RPT	PENDING	NULL	NULL	NULL	NULL
wo_01HZ...H	WIK	SUPPORT	HS1	ASSIGNED	TEAM	ctr_Z071H (internal)	NULL (awaiting claim)	NULL
wo_01HZ...J	WIK	INSTALLATION	H01	ASSIGNED	TEAM	ctr_ADRIAN (external)	NULL (awaiting dispatcher ack)	NULL

5.2 wo_notes

Append-only typed log of notes attached to a WO. The framework validates the JSONB payload against the registered note_kind schema at write time.

Column	Type	Notes
note_id	TEXT PK	ULID
wo_id	TEXT NOT NULL	FK to work_order
note_kind	TEXT NOT NULL	FK to wo_note_kind_registry(note_kind) for the WO's operator. Example: findings, optical_readings
content_text	TEXT NULL	Free text (used when the note kind has NULL schema in registry).
content_jsonb	JSONB NULL	Structured payload (used when the note kind has a JSON schema). Mutually exclusive with content_text.
appended_at	TIMESTAMP NOT NULL DEFAULT NOW()	
appended_by	TEXT NOT NULL	Actor — agent ID, worker ID, or system ID
appended_by_role	TEXT NOT NULL	CSR, DISPATCHER, CONTRACTOR, FIELD_TECH, NOC, SYSTEM_WORKER
superseded_by_note_id	TEXT NULL	When the note kind is non-append-only (e.g., customer_appointment) and updated, the older row is marked superseded by the newer one.

Indexes: PK; (wo_id, appended_at DESC); (wo_id, note_kind); partial (wo_id, note_kind) WHERE superseded_by_note_id IS NULL. P10Y retention.

Sample data:

note_id	wo_id	note_kind	content_text / content_jsonb	appended_by_role	appended_at
note_01HZ_001	wo_01HZ...A	csr_note	LOS FLASHING RED - MISSING CHANNELS - ATHI RIVER ROAD 104	CSR	2026-05-18 09:00:33
note_01HZ_002	wo_01HZ...A	customer_appointment	jsonb: {"scheduledAt":"2026-05-18T14:00:00Z","comments":"Customer available 2-5pm"}	DISPATCHER	2026-05-18 09:15:02
note_01HZ_003	wo_01HZ...A	findings	Cable modem rebooted, DOCSIS signal locked, TV channels restored	FIELD_TECH	2026-05-18 14:45:11
note_01HZ_004	wo_01HZ...A	solution	Faulty data cable replaced from drop point to indoor unit	FIELD_TECH	2026-05-18 14:46:33
note_01HZ_005	wo_01HZ...A	hfc_signal_readings	jsonb: {"downstreamLevel":-3.2,"upstreamLevel":42.1,"snr":38.5,"package":"INET_50M"}	FIELD_TECH	2026-05-18 14:47:01
note_01HZ_006	wo_01HZ...A	final_reason_set	jsonb: {"finalReasonCode":"CABLE_REPLACED"}	CONTRACTOR	2026-05-18 14:50:00
note_01HZ_007	wo_01HZ...D	dispatcher_note	Customer relocating to Plot 47, Westlands. GSR scheduled for 2026-05-19	DISPATCHER	2026-05-18 10:00:00

5.3 `wo_attachments`

Append-only file attachments. Storage URI is opaque to the framework — v1.0 stores files in MinIO/S3-compatible storage.

Column	Type	Notes
attachment_id	TEXT PK	ULID
wo_id	TEXT NOT NULL	FK to work_order
category	TEXT NOT NULL	Operator-defined. Examples: setup_photo, rf_optical_reading, speed_test, damaged_equipment, signed_handover
description	TEXT NOT NULL	Free text
storage_uri	TEXT NOT NULL	Opaque to framework. Example: s3://sophix-wo-attachments/wik/2026/05/18/wo_01HZ_A/photo_001.jpg
mime_type	TEXT NOT NULL	Example: image/jpeg
file_size_bytes	BIGINT NOT NULL	
attached_at	TIMESTAMP NOT NULL DEFAULT NOW()	
attached_by	TEXT NOT NULL	
attached_by_role	TEXT NOT NULL	Same enum as wo_notes.appended_by_role

Indexes: PK; (wo_id, attached_at DESC); (wo_id, category).

Sample data:

attachment_id	wo_id	category	description	storage_uri	attached_at
att_01HZ_001	wo_01HZ...A	setup_photo	ONT mounted on living-room wall after install	s3://.../photo_001.jpg	2026-05-18 14:30:00
att_01HZ_002	wo_01HZ...A	rf_optical_reading	Easydocsis CMTS screen showing modem online	s3://.../easydocsis_001.png	2026-05-18 14:35:00

attachment_id	wo_id	category	description	storage_uri	attached_at
att_01HZ_003	wo_01HZ...A	speed_test	Ookla speed test showing 50/10 Mbps achieved	s3://.../speedtest_001.png	2026-05-18 14:38:00

5.4 wo_status_history

One row per state transition.

Column	Type	Notes
id	BIGSERIAL PK	
wo_id	TEXT NOT NULL	FK to work_order
operator_code	TEXT NOT NULL	Denormalized for index efficiency
previous_status	TEXT NULL	NULL on first transition (creation)
new_status	TEXT NOT NULL	
previous_phase	TEXT NULL	
new_phase	TEXT NULL	
transition_at	TIMESTAMP NOT NULL DEFAULT NOW()	
triggered_by	TEXT NOT NULL	wo_created, manual_assign, auto_assign_worker, manual_start, finalize_first_confirm, finalize_second_confirm, cancel_api, phase_transition, reassign
triggered_by_detail	TEXT NULL	Actor / worker / timer ID
metadata	JSONB NULL	Optional extension

Indexes: PK; (wo_id, transition_at); (operator_code, transition_at DESC). P10Y retention. Monthly partitioned.

5.5 wo_assignment_history

Reassignment audit trail. Distinct from wo_status_history because reassignments don't always change status (Pattern B: in-progress install reassigned to maintenance team, status stays IN_PROGRESS).

Column	Type	Notes
id	BIGSERIAL PK	
wo_id	TEXT NOT NULL	FK
previous_contractor_id	TEXT NULL	
previous_team_id	TEXT NULL	
previous_technician_id	TEXT NULL	
new_contractor_id	TEXT NULL	
new_team_id	TEXT NULL	
new_technician_id	TEXT NULL	
assignment_reason	TEXT NOT NULL	initial_assignment, manual_reassignment, escalation_pattern_b, dispatcher_correction, auto_assignment
assigned_by	TEXT NOT NULL	Actor or worker
assigned_at	TIMESTAMP NOT NULL DEFAULT NOW()	

Indexes: PK; (wo_id, assigned_at DESC). P10Y retention.

5.6 wo_audit_log

Catch-all for actions that aren't status or assignment changes: note appended, attachment added, PDF print, manual edit of scheduled time, etc.

Column	Type	Notes
id	BIGSERIAL PK	
wo_id	TEXT NOT NULL	

Column	Type	Notes
operator_code	TEXT NOT NULL	Denormalized for index efficiency
action	TEXT NOT NULL	wo_created, note_appended, attachment_added, scheduled_at_updated, printed, reassigned, status_changed, phase_transitioned, finalize_first_confirm, finalize_second_confirm, cancelled, finalize_reverted
actor	TEXT NOT NULL	
actor_role	TEXT NOT NULL	
action_detail	JSONB NULL	Action-specific payload. Example for printed: {"printedTo":"PDF","printFormat":"Laser","copyNumber":3}
occurred_at	TIMESTAMP NOT NULL DEFAULT NOW()	

Indexes: PK; (wo_id, occurred_at DESC); (operator_code, action, occurred_at DESC). P10Y retention. Monthly partitioned.

6. Contractor + team + technician + skills model

The framework owns the lightweight registry for contractors, teams, technicians, and skills. v1.0 ships the schema seeded for KE WIK; full management UI is dispatcher-tool concern (BO UI).

6.1 wo_contractor

Column	Type	Notes
contractor_id	TEXT PK	Example: ctr_Z071H
operator_code	TEXT NOT NULL	
contractor_name	TEXT NOT NULL	Example: Z071H In-House Support
slot_label	TEXT NOT NULL	Visible to dispatcher when assigning
tech_regions_served	JSONB NOT NULL	Array of tech region codes. Example: ["NRB-KAREN","NRB-LANGATA","NRB-LAVINGTON"]
active	BOOLEAN NOT NULL DEFAULT true	
is_internal	BOOLEAN NOT NULL DEFAULT false	true = Wananchi internal team (NOC, maintenance); false = external contractor
created_at	TIMESTAMP NOT NULL	

Indexes: PK; (operator_code, active); GIN on tech_regions_served.

Sample data (v1.0 WIK seed, partial):

contractor_id	contractor_name	slot_label	tech_regions_served	is_internal
ctr_Z071H	Z071H In-House Support	Z071H In-House Support	["NRB-KAREN","NRB-LANGATA","NRB-LAVINGTON"]	true
ctr_GTECH	GTECH Provision & Updates	GTECH Provision & Updates	["NRB-WESTLANDS","NRB-PARKLANDS"]	false
ctr_ADRIAN	Adrian Provision Group	Adrian Provision Group	["NRB-EASTLANDS","NRB-KASARANI"]	false
ctr_NOC_MAINT	NOC Maintenance Team	NOC Maintenance	["ALL"]	true

6.2 wo_team

Column	Type	Notes
team_id	TEXT PK	Example: team_rapid_response
contractor_id	TEXT NOT NULL	FK
team_name	TEXT NOT NULL	Example: Rapid Response Team
active	BOOLEAN NOT NULL DEFAULT true	

6.3 wo_technician

Individual technician registry. **Population is mandatory for internal contractor teams** (Wananchi in-house staff have BO logins, payroll IDs, known skills) and **optional for external contractor teams** (most external contractors work with rotating freelance staff and don't track them in SOPHIX; their team gets the WO and their own dispatcher distributes work through their own channels). An external contractor MAY still register specific named technicians if it makes sense — e.g., a long-tenured freelancer they always trust with a specific job_type — but the framework does not require it.

When an external team has no registered technicians, all assignments to that team MUST be TEAM mode (no `technicianId` to supply). See §6.6.

Column	Type	Notes
`technician_id`	TEXT PK	Example: `tech_01HZ_KAREN_03`
`team_id`	TEXT NOT NULL	FK to `wo_team`
`display_name`	TEXT NOT NULL	Example: `Adrien Modicom 3`
`tech_region_code`	TEXT NOT NULL	Single primary region. Example: `NRB-KAREN`
`active`	BOOLEAN NOT NULL DEFAULT true	

6.4 wo_technician_skill

A technician's skills determine which WO job types they can be assigned in TECHNICIAN mode (workshop: "a tech without the skill for a Job Type does not appear as a slot option"). Applies in practice only to **internal contractor teams** (since external teams typically have no registered technicians per §6.3); for external teams the skill match is the contractor's own responsibility — the WO module records team-level assignment without skill validation at the technician layer.

Column	Type	Notes
`id`	BIGSERIAL PK	
`technician_id`	TEXT NOT NULL	FK
`service_plan_type`	TEXT NULL	`INT` / `OTH` / `PHN` / `VID`. NULL = applies to all
`work_type_code`	TEXT NULL	A specific job_type code from `wo_job_type_catalog`. NULL = all work types for the service plan

Indexes: PK; `(technician_id)`.

6.5 wo_schedule

Day-of-week availability with working-hours window, scoped to either a team or an individual technician. One unified table replaces what would otherwise be separate per-scope schedule tables.

A team-scoped row defines the working hours of the team as a whole (e.g., `team_gtech_provision_westlands` operates Mon–Sat 09–18). A technician-scoped row overrides for that individual (e.g., `tech_01HZ_KAREN_03` does not work Wednesday, or works half-day Saturday). Resolution at assignment/claim time:

1. If TECHNICIAN mode: check the technician's own `wo_schedule` row for the day; if no per-tech row exists, fall back to the team's row; if neither exists → tech not available that day.
2. If TEAM mode: check the team's `wo_schedule` row for the day; if no row → team not available that day.

Column	Type	Notes
`id`	BIGSERIAL PK	
`scope`	TEXT NOT NULL	`TEAM` or `TECHNICIAN`
`scope_ref`	TEXT NOT NULL	team_id when scope=TEAM; technician_id when scope=TECHNICIAN
`day_of_week`	INTEGER NOT NULL	1 = Monday … 7 = Sunday
`start_hour`	INTEGER NOT NULL	0–23. Example: 9 = 09h
`end_hour`	INTEGER NOT NULL	1–24. Example: 18 = 18h. Exclusive upper bound. A scheduled_at hour H is in-window iff `start_hour ≤ H < end_hour`

UNIQUE `(scope, scope_ref, day_of_week)` — at most one window per (scope, ref, day). Indexes: PK; UNIQUE constraint; `(scope_ref, scope)` for fast lookup by team_id or tech_id.

Sample data:

id	scope	scope_ref	day_of_week	start_hour	end_hour	meaning
1	TEAM	team_gtech_provision_westlands	1 (Mon)	9	18	team default Mon
2	TEAM	team_gtech_provision_westlands	2 (Tue)	9	18	team default Tue
3	TEAM	team_gtech_provision_westlands	3 (Wed)	9	18	team default Wed
4	TEAM	team_gtech_provision_westlands	4 (Thu)	9	18	team default Thu
5	TEAM	team_gtech_provision_westlands	5 (Fri)	9	18	team default Fri
6	TEAM	team_gtech_provision_westlands	6 (Sat)	9	13	team default Sat half day
7	TECHNICIAN	tech_01HZ_KAREN_03	6 (Sat)	0	0	Adrien Modicom 3 NOT available Sat (window 00–00 = closed)
8	TECHNICIAN	tech_01HZ_KAREN_05	1 (Mon)	13	20	half-day-late Mon for Modicom 5 (overrides team window)

For technician `tech_01HZ_KAREN_03`: Mon–Fri picks up team window 09–18; Sat resolves to per-tech row 00–00 (not available); Sun no row anywhere → not available.

For technician `tech_01HZ_KAREN_05`: Mon resolves to per-tech 13–20 (afternoon start); Tue–Fri picks up team 09–18; Sat picks up team 09–13; Sun no row → not available.

For TEAM-mode WO assignment, framework checks only the team-scope row and ignores per-tech overrides (since claim happens later, and the claiming tech's own window is checked at claim time).

6.6 Assignment workflow

The framework supports two assignment modes — **TECHNICIAN** (specific tech named at assignment) and **TEAM** (team named; ownership left open). The claim flow that follows TEAM-mode assignment has two variants depending on whether the team belongs to an internal or external contractor.

TECHNICIAN-mode assignment. Only valid when the named technician exists in `wo_technician` and the technician's `team_id` matches the WO's `assigned_team_id`. In practice this is used for internal teams (where every tech is registered) and the rare external case where the contractor has registered a specific freelancer. `assigned_technician_id` is set at assignment time; `claimed_at = assigned_at`; no claim step is needed.

TEAM-mode assignment — internal team. Framework leaves `assigned_technician_id = NULL`. Every active technician on the team is notified ("WO awaiting claim"). The first technician to call `POST /api/work-orders/{id}/claim` with their own `technicianId` wins via atomic DB update; `assigned_technician_id` is set, `claimed_at` recorded, an event is emitted, the other team members are notified that the WO has been claimed. This is the classic race-to-claim flow.

TEAM-mode assignment — external team. Framework leaves `assigned_technician_id = NULL`. Notification goes to the contractor's shared dispatcher inbox (email / Slack / WhatsApp via the integration channels the external contractor has agreed to). The contractor's dispatcher acknowledges receipt by calling `POST /api/work-orders/{id}/claim` with the flag `claimedByExternalDispatcher: true` and their BO user as `claimedBy`. Framework records `claimed_at = NOW()` and `claimed_by_dispatcher = <BO user>` while keeping `assigned_technician_id = NULL` permanently. We never know which freelancer the external contractor sends; we know which Wananchi-side dispatcher (or contractor manager logged in to BO UI) acknowledged the WO. The WO can then `/start`, accumulate notes + attachments + finalize like any other WO; all entries in `wo_notes` and `wo_attachments` carry the dispatcher user as `appended_by`/`attached_by` since field-tech identity isn't tracked.

Mode is set automatically by the framework based on what's supplied in `/assign`:

- `technicianId` provided → mode = `TECHNICIAN`, `claimed_at = assigned_at`, `assigned_technician_id` set immediately.
- `technicianId` omitted → mode = `TEAM`, `claimed_at` NULL, `assigned_technician_id` NULL until a `/claim` call sets it (internal tech path) or sets `claimed_by_dispatcher` (external dispatcher path).

Manual assignment — dispatcher calls `POST /api/work-orders/{id}/assign`:

```
// TECHNICIAN mode (specific tech named):
{
  "contractorId":    "ctr_Z071H",
  "teamId":          "team_rapid_response",
  "technicianId":    "tech_01HZ_KAREN_03",
  "scheduledAt":     "2026-05-18T14:00:00Z",
  "assignmentReason":"manual_assignment"
}

// TEAM mode (team picked; claim flow chosen at claim time):
{
  "contractorId":    "ctr_GTECH",
  "teamId":          "team_gtech_provision_westlands",
  "technicianId":    null,
  "scheduledAt":     "2026-05-18T14:00:00Z",
  "assignmentReason":"manual_team_assignment"
}
```

Framework validates:

- In TECHNICIAN mode: the named technician exists, has `team_id = assigned_team_id`, has the required skills for the WO's `job_type_code` (via `wo_technician_skill`), and the schedule resolution (per-tech override or team default in `wo_schedule`) places the `scheduled_at` hour inside an open window. The technician must not already own a non-terminal WO (per R-WO-FW-11e); if they do, returns 400 `TECHNICIAN_HAS_PENDING_WO` with the conflicting `woId`. Reject with 400 on any miss.
- In TEAM mode: the team's `wo_schedule` row for the scheduled day exists and contains the scheduled hour. The contractor's `tech_regions_served` contains the WO's `tech_region_code`. Skill validation is NOT performed at the framework level (responsibility delegated to the team's dispatcher / contractor's own scheduling logic). The tech-load check happens later, at internal-tech `/claim` time — not at TEAM-mode assignment time.

Auto-assignment — the BPMN places a `wo-auto-assign-contractor` external task. The worker reads the operator's Drools KJAR (`wo_flow_config.drools_kjar_ref`), invokes it with the WO context, gets back a `ContractorAssignmentDecision` with the picked `{contractorId, teamId, technicianId?}` (technicianId optional — Drools may leave it NULL for team-mode). Persists the decision; transitions status to ASSIGNED. v1.0 Drools rules are simple (first match by region + availability; for internal teams in TECHNICIAN mode, also skill-match); future iterations refine. Operator config (per-flow) decides whether auto-assignment prefers TEAM or TECHNICIAN mode for a given job_type — captured in the Drools rules, not in framework schema.

Claim (internal technician path) — `POST /api/work-orders/{id}/claim` with `{technicianId, claimedAt}` and `claimedByExternalDispatcher: false` (default). Framework validates: WO is in status ASSIGNED; `assignment_mode = TEAM`; tech exists in `wo_technician` with matching `team_id`; tech has the required skills + per-tech schedule window; tech does not already own a non-terminal WO (per R-WO-FW-11e). On pass: `assigned_technician_id` set, `claimed_at` set, audit-logged, `WorkOrderClaimedByTechnician` event emitted; other team members' claim-prompt notifications are cancelled.

Claim (external dispatcher path) — `POST /api/work-orders/{id}/claim` with `{claimedByExternalDispatcher: true, claimedBy: <BO user>, claimedAt}`. Framework validates: WO is in status ASSIGNED; `assignment_mode = TEAM`. On pass: `claimed_by_dispatcher` set, `claimed_at` set, `assigned_technician_id` stays NULL, audit-logged, `WorkOrderClaimedByDispatcher` event emitted.

If two claims race (whether internal-tech or external-dispatcher), framework relies on DB-level atomic update against `claimed_at IS NULL`. Second claim returns 409 `ALREADY_CLAIMED` with the winning party recorded.

Reassignment — `POST /api/work-orders/{id}/reassign` with new `{contractorId, teamId, technicianId?, reason}`. Status does NOT change. Resets to the supplied mode: if `technicianId` is in the new payload, mode → TECHNICIAN; if omitted, mode → TEAM and `claimed_at` / `claimed_by_dispatcher` cleared. Writes `wo_assignment_history` row. Emits `WorkOrderReassigned` event. This is the framework primitive for Pattern B (Installation passed to maintenance — same WO, new team, possibly with different mode).

Start — `POST /api/work-orders/{id}/start` requires `claimed_at IS NOT NULL` (either via real-tech claim or via external dispatcher ack). In TECHNICIAN mode this is satisfied immediately after `/assign`. In TEAM mode the tech or the external dispatcher must first call `/claim`. If `/start` is called while still unclaimed, framework returns 400 `WORK_ORDER_NOT_CLAIMED`.

7. Kafka event catalog

The framework emits events at every state transition + key actions. All on topic `sophix.work-orders.*`. FUL-02 already depends on `WorkOrderFinalized` + `WorkOrderCancelled`; new consumers (Ticketing, OSR-*, NOC ops dashboards) will subscribe to others.

Event type	Topic	Emitted when	Key consumers
`WorkOrderCreated`	`sophix.work-orders.created`	After WO row + Camunda instance both written	NOC dashboards, audit
`WorkOrderAssigned`	`sophix.work-orders.assigned`	Status → ASSIGNED	NOC, audit
`WorkOrderClaimedByTechnician`	`sophix.work-orders.claimed-by-tech`	TEAM-mode internal-team WO claimed via `/claim` by a registered technician — `assigned_technician_id` set	NOC, notification (cancel other team members' alerts), audit
`WorkOrderClaimedByDispatcher`	`sophix.work-orders.claimed-by-dispatcher`	TEAM-mode external-team WO acknowledged via `/claim` with `claimedByExternalDispatcher=true` — `claimed_by_dispatcher` set	NOC, audit
`WorkOrderReassigned`	`sophix.work-orders.reassigned`	`/reassign` API called; no status change	NOC, audit, SLA tracker
`WorkOrderDispatched`	`sophix.work-orders.dispatched`	`assigned_technician_id` first set	NOC, audit
`WorkOrderInProgress`	`sophix.work-orders.in-progress`	Status → IN_PROGRESS	NOC, audit
`WorkOrderPhaseTransitioned`	`sophix.work-orders.phase-transitioned`	`current_phase` changes	Shifting consumers, audit
`WorkOrderNoteAppended`	`sophix.work-orders.note-appended`	New row in `wo_notes`	Audit; future field-team comms
`WorkOrderAttachmentAdded`	`sophix.work-orders.attachment-added`	New row in `wo_attachments`	Audit
`WorkOrderFinalizationPending`	`sophix.work-orders.finalization-pending`	First-confirm finalize	Audit, dispatcher UI
`WorkOrderFinalized`	`sophix.work-orders.finalized`	Second-confirm finalize succeeds; status → COMPLETED	**FUL-02 STEP-INSTALL** (already consumes), Ticketing (future), audit, Finance (contractor invoicing)
`WorkOrderEscalationCandidate`	`sophix.work-orders.escalation-candidate`	Finalization with `triggers_escalation_event = true` final reason	**Ticketing (future)** — decides whether to spawn escalation WO
`WorkOrderCancelled`	`sophix.work-orders.cancelled`	Status → CANCELLED	**FUL-02 STEP-INSTALL** (already consumes), audit

Sample event payload — `WorkOrderFinalized`:

```
{
  "eventType":   "WorkOrderFinalized",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp":   "2026-05-18T14:50:00.789Z",
  "payload": {
    "woId":                    "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "operatorCode":            "WIK",
    "kind":                    "INSTALLATION",
    "jobTypeCode":             "H01",
    "networkType":             "HFC",
    "customerId":              "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "homepassId":              "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "originatingContextType":  "ORDER",
    "originatingContextRef":   "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "assignedContractorId":    "ctr_Z071H",
    "assignedTeamId":          "team_rapid_response",
    "assignedTechnicianId":    "tech_01HZ_KAREN_03",
    "assignmentMode":          "TECHNICIAN",
    "claimedByDispatcher":     null,
    "claimedAt":               "2026-05-18T09:15:00.012Z",
    "finalReasonCode":         "INSTALL_COMPLETED_CUSTOMER_UP",
    "triggersEscalation":      false,
    "createdAt":               "2026-05-18T09:00:00.123Z",
    "completedAt":             "2026-05-18T14:50:00.789Z",
    "durationSeconds":         20999
  }
}
```

Sample event payload — `WorkOrderEscalationCandidate`:

```
{
```

```
  "eventType":   "WorkOrderEscalationCandidate",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
  "timestamp":   "2026-05-18T16:30:00Z",
  "payload": {
    "woId":             "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
    "operatorCode":     "WIK",
    "kind":             "SUPPORT",
    "jobTypeCode":      "GP3",
    "finalReasonCode":  "COULD_NOT_RESOLVE_ESCALATED",
    "lastFindings":     "Optical ONT RX power -32 dBm — out of acceptable range",
    "customerId":       "cust_01HX...",
    "homepassId":       "hp_01HX..."
  }
}
```

Ticketing module (future) consumes this and decides whether to spawn a QCS WO with `master_wo_id = wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2`, `link_type = ESCALATION`, candidate-group routed to NOC maintenance.

8. REST API surface

The framework exposes a consumer-facing REST API. Same API serves FUL-02 (machine consumer), Ticketing (future), OSR-* (future), Dispatcher BO UI (human consumer), and the contractor portal.

8.1 `POST /api/work-orders` — Create

```
POST /api/work-orders HTTP/1.1
Host: work-order.sophix.internal
Authorization: Bearer <module-svc-creds OR user JWT>
Content-Type: application/json

{
  "operatorCode":            "WIK",
  "kind":                    "INSTALLATION",
  "jobTypeCode":             "H01",
  "customerId":              "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "homepassId":              "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "originatingContextType":  "ORDER",
  "originatingContextRef":   "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "idempotencyKey":          "ful-02-step-install-ord_01HZ...A",
  "masterWoId":              null,
  "linkType":                null,
  "metadata":                {"agentSalesId":"agent_01HZ..."}
}
```

Response 201:

```
{
  "woId":              "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "status":            "PENDING",
  "processInstanceId": "cam_pi_01HZ...",
  "createdAt":         "2026-05-18T09:00:00.345Z"
}
```

Errors:

Status	When
400 `INVALID_OPERATOR`	`operatorCode` not in v1.0 enum
400 `UNKNOWN_KIND`	`kind` not in `{INSTALLATION, SUPPORT, SHIFTING}`
400 `UNKNOWN_JOB_TYPE`	`(operatorCode, jobTypeCode)` not in `wo_job_type_catalog`
400 `BPMN_NOT_DEPLOYED`	No `wo_flow_config` row for `(operatorCode, kind)`
400 `INVALID_LINKAGE`	`masterWoId` provided but `linkType` missing (or vice versa)
409 `IDEMPOTENCY_HIT`	Existing WO with same `(operatorCode, originatingContextType, idempotencyKey)` — returns the existing row with `idempotencyHit: true`

8.2 `GET /api/work-orders/{woId}` — Read

Returns the full WO with notes, attachments, status history, assignment history. Used by dispatcher BO UI and by audit consumers.

```
GET /api/work-orders/wo_01HZ... HTTP/1.1
Authorization: Bearer <creds>
```

Response 200:

```
{
  "woId":              "wo_01HZ...",
  "operatorCode":      "WIK",
  "kind":              "INSTALLATION",
  "jobTypeCode":       "H01",
  "status":            "IN_PROGRESS",
  "currentPhase":      null,
  "customerId":        "cust_01HX...",
  "homepassId":        "hp_01HX...",
  "assignedContractorId": "ctr_Z071H",
  "assignedTeamId":    "team_rapid_response",
  "assignedTechnicianId": "tech_01HZ_KAREN_03",
  "scheduledAt":       "2026-05-18T14:00:00Z",
  "createdAt":         "2026-05-18T09:00:00Z",
  "assignedAt":        "2026-05-18T09:15:00Z",
  "inProgressAt":      "2026-05-18T14:30:00Z",
  "notes":             [/* see 5.2 sample data */],
  "attachments":       [/* see 5.3 sample data */],
  "statusHistory":     [/* see 5.4 */],
  "assignmentHistory": [/* see 5.5 */]
}
```

8.3 `POST /api/work-orders/{woId}/assign` — Manual assignment

Two modes — supplying `technicianId` means TECHNICIAN mode (specific tech owns the WO immediately); omitting it means TEAM mode (any tech on the team can claim).

TECHNICIAN mode:

```
POST /api/work-orders/wo_01HZ.../assign HTTP/1.1

{
  "contractorId":    "ctr_Z071H",
  "teamId":          "team_rapid_response",
  "technicianId":    "tech_01HZ_KAREN_03",
  "scheduledAt":     "2026-05-18T14:00:00Z",
  "assignmentReason":"manual_assignment"
}
```

Status → ASSIGNED, `assignment_mode = TECHNICIAN`, `assigned_technician_id` set, `claimed_at = assigned_at`. Emits `WorkOrderAssigned`.

TEAM mode:

```
POST /api/work-orders/wo_01HZ.../assign HTTP/1.1

{
  "contractorId":    "ctr_Z071H",
  "teamId":          "team_rapid_response",
  "technicianId":    null,
  "scheduledAt":     "2026-05-18T14:00:00Z",
  "assignmentReason":"manual_team_assignment"
}
```

Status → ASSIGNED, `assignment_mode = TEAM`, `assigned_technician_id` remains NULL, `claimed_at` NULL. Emits `WorkOrderAssigned`. Notification fires to every active technician on `team_rapid_response`: "WO wo_01HZ... available for claim."

Validation per §6.6 (region, skill, schedule).

8.4 `POST /api/work-orders/{woId}/reassign` — Reassignment without closure

Same body shape as `/assign` (both modes supported). Status does not change; `wo_assignment_history` row written; `WorkOrderReassigned` emitted. Mode may change between assignments — e.g., a TEAM-mode WO can be reassigned as TECHNICIAN-mode to override the claim mechanism.

8.5 `POST /api/work-orders/{woId}/claim` — Claim a TEAM-mode WO

Two paths — same endpoint, different request body:

Path A: Internal technician claim (real tech registered in `wo_technician`)

```
POST /api/work-orders/wo_01HZ.../claim HTTP/1.1

{
  "claimedByExternalDispatcher": false,
  "technicianId":                "tech_01HZ_KAREN_05",
  "claimedAt":                   "2026-05-18T13:42:00Z"
}
```

Framework:

1. Verifies status = ASSIGNED and `assignment_mode = TEAM`.
2. Verifies the technician exists in `wo_technician` and `team_id` matches the WO's `assigned_team_id`.
3. Verifies the technician has the required skills for the WO's `job_type_code` (per §6.4).
4. Verifies the technician's schedule (resolved per §6.5: per-tech row overrides team default) covers the WO's `scheduled_at`.
5. Atomically updates: `UPDATE work_order SET assigned_technician_id = $1, claimed_at = NOW() WHERE wo_id = $2 AND claimed_at IS NULL`.
6. Emits `WorkOrderClaimedByTechnician`.
7. Signals notification module to cancel the rest of the team's claim-prompt notifications.

Path B: External-team dispatcher acknowledgment (no individual tech tracked)

```
POST /api/work-orders/wo_01HZ.../claim HTTP/1.1

{
  "claimedByExternalDispatcher": true,
  "claimedBy":                   "dispatcher_user_gtech_01",
  "claimedAt":                   "2026-05-18T13:42:00Z"
}
```

Framework:

1. Verifies status = ASSIGNED and `assignment_mode = TEAM`.
2. Verifies `claimedBy` is a valid BO user authorised to act for the WO's `assigned_contractor_id`.
3. Atomically updates: `UPDATE work_order SET claimed_by_dispatcher = $1, claimed_at = NOW() WHERE wo_id = $2 AND claimed_at IS NULL`. `assigned_technician_id` stays NULL.
4. Emits `WorkOrderClaimedByDispatcher`.

Response 200 for either path: returns updated WO.

Errors:

Status	When
400 `WO_NOT_IN_TEAM_MODE`	WO is not in TEAM mode (e.g., already in TECHNICIAN mode or status not ASSIGNED)
400 `TECHNICIAN_NOT_IN_TEAM`	Path A only — the technician doesn't belong to the assigned team
400 `TECHNICIAN_SKILL_MISMATCH`	Path A only — tech doesn't have the required skill for `job_type_code`
400 `TECHNICIAN_NOT_AVAILABLE`	Path A only — tech's schedule doesn't cover `scheduled_at`
400 `TECHNICIAN_HAS_PENDING_WO`	Path A only — tech already owns a non-terminal WO (per R-WO-FW-11e); error body includes the conflicting `woId`
400 `DISPATCHER_NOT_AUTHORISED`	Path B only — `claimedBy` is not a recognized BO user for `assigned_contractor_id`
409 `ALREADY_CLAIMED`	Race lost. Response includes the winning party (`claimedBy` for Path B winners, `technicianId` for Path A winners) and `claimedAt`.

8.6 `POST /api/work-orders/{woId}/start` — Contractor confirms work has begun

```
POST /api/work-orders/wo_01HZ.../start HTTP/1.1

{
  "startedBy": "tech_01HZ_KAREN_03"
}
```

Status PENDING/ASSIGNED → IN_PROGRESS. Sets `in_progress_at`. Emits `WorkOrderInProgress`.

Requires `claimed_at IS NOT NULL` (claim has happened via either internal-tech path or external-dispatcher path). In TECHNICIAN mode this is satisfied automatically at assignment. In TEAM mode the tech or external dispatcher must first call `/claim`. Otherwise returns 400 `WORK_ORDER_NOT_CLAIMED`.

8.7 `POST /api/work-orders/{woId}/transition-phase` — Multi-phase transition

```
POST /api/work-orders/wo_01HZ.../transition-phase HTTP/1.1
```

```
{
  "newPhase":      "RECONNECT",
  "transitionedBy":"csr_user_42"
}
```

Status stays IN_PROGRESS; `current_phase` updates; `WorkOrderPhaseTransitioned` event. Used by Shifting flow (and any future multi-phase flow). Validation: new phase must be a legal next phase per the BPMN's phase definition.

8.8 `POST /api/work-orders/{woId}/notes` — Append a note

```
POST /api/work-orders/wo_01HZ.../notes HTTP/1.1

{
  "noteKind":      "optical_readings",
  "contentJsonb":  {
    "oltPort":      "0/11/1/6",
    "onuId":        12,
    "distanceM":    1200,
    "opticalTempC": 40,
    "ontRxDbm":     -23.92,
    "ontTxDbm":     2.01,
    "oltRxDbm":     79.54,
    "ip":           "10.1.42.27",
    "lastowncause": "dying-gasp",
    "batt":         "notsupport"
  },
  "appendedBy":     "tech_01HZ_KAREN_03",
  "appendedByRole": "FIELD_TECH"
}
```

Validation: `noteKind` must exist in `wo_note_kind_registry` for the WO's operator. If the registry row has a `schema_jsonb`, the request's `contentJsonb` must validate against it (else 400 `NOTE_SCHEMA_VIOLATION`). If schema is NULL, the request must use `contentText` (free text).

Response 201: returns the created note.

8.9 `POST /api/work-orders/{woId}/attachments` — Add an attachment

```
POST /api/work-orders/wo_01HZ.../attachments HTTP/1.1
Content-Type: multipart/form-data

(file binary)
+ category: setup_photo
+ description: ONT mounted on living-room wall after install
+ attachedBy: tech_01HZ_KAREN_03
+ attachedByRole: FIELD_TECH
```

Framework uploads file to MinIO/S3, records URI + metadata in `wo_attachments`, emits `WorkOrderAttachmentAdded`.

Response 201:

```
{
  "attachmentId": "att_01HZ_001",
  "storageUri":   "s3://sophix-wo-attachments/wik/2026/05/18/wo_01HZ_A/photo_001.jpg",
  "attachedAt":   "2026-05-18T14:30:00Z"
}
```

8.10 `POST /api/work-orders/{woId}/finalize-first-confirm` — Step 1 of 2-step finalize

```
POST /api/work-orders/wo_01HZ.../finalize-first-confirm HTTP/1.1

{
  "finalReasonCode": "INSTALL_COMPLETED_CUSTOMER_UP",
  "confirmedBy":     "ctr_Z071H_dispatcher_01"
}
```

Framework:

1. Verifies WO is in IN_PROGRESS.
2. Writes the `final_reason_set` note kind with the chosen reason.
3. Sets `final_reason_code` on `work_order`.
4. Status → FINALIZATION_PENDING.
5. Emits `WorkOrderFinalizationPending`.

Does NOT run the checklist yet. Dispatcher can still revert if they realize something's missing.

8.11 `POST /api/work-orders/{woId}/finalize-second-confirm` — Step 2 of 2-step finalize

No body. Framework:

1. Verifies WO is in FINALIZATION_PENDING.
2. Reads `wo_finalization_requirements` for `(operatorCode, kind, jobTypeCode)`. Specific job-type row overrides default (`job_type_code=NULL`) row.
3. Counts notes per kind and attachments per category on this WO.
4. Verifies every required note kind has ≥1 entry; every required attachment category has ≥ the configured minimum count.
5. If miss → 400 `FINALIZATION_CHECKLIST_FAILED` with array of missing items:

```
{
  "error": "FINALIZATION_CHECKLIST_FAILED",
  "missing": {
    "noteKinds":           ["hfc_signal_readings"],
    "attachmentCategories":["speed_test"]
  }
}
```

WO stays in FINALIZATION_PENDING; dispatcher fixes; retries.

6. If pass → status → COMPLETED, `completed_at` = NOW(), emit `WorkOrderFinalized`. Also emit `WorkOrderEscalationCandidate` if the final reason has `triggers_escalation_event = true`.

Response 200:

```
{
  "woId":              "wo_01HZ...",
  "status":            "COMPLETED",
  "completedAt":       "2026-05-18T14:50:00Z",
  "triggersEscalation":false
}
```

8.12 `POST /api/work-orders/{woId}/finalize-revert` — Revert from FINALIZATION_PENDING

```
POST /api/work-orders/wo_01HZ.../finalize-revert HTTP/1.1

{
  "revertedBy":  "ctr_Z071H_dispatcher_01",
  "reason":      "Missing speed test attachment — recovering"
}
```

Status FINALIZATION_PENDING → IN_PROGRESS. Audit-logged. Used when dispatcher catches a mistake before second-confirm.

8.13 `POST /api/work-orders/{woId}/cancel` — Cancel

```
POST /api/work-orders/wo_01HZ.../cancel HTTP/1.1

{
  "cancellationReason": "CUSTOMER_DECLINED",
  "cancellationDetail": "Customer requested cancellation — moved to competitor",
  "cancelledBy":        "csr_user_42"
}
```

Valid from PENDING, ASSIGNED, IN_PROGRESS, or FINALIZATION_PENDING. Framework:

1. Sets `status = CANCELLED`, `cancelled_at = NOW()`, writes reason fields.
2. Sends Camunda message `WorkOrderCancellationRequested` to interrupt the BPMN.
3. Emits `WorkOrderCancelled`.

8.14 `GET /api/work-orders/{woId}/print-pdf` — Render WO PDF for field tech

Returns a generated PDF rendering of the WO (client address, scheduled time, contact details, job type, customer notes from CSR, dispatcher notes). One PDF at a time per workshop's print pattern. Audit-logged as `printed` action with copy number incremented.

```
GET /api/work-orders/wo_01HZ.../print-pdf HTTP/1.1
```

```
Accept: application/pdf
```

Response 200: `application/pdf` binary.

8.15 `GET /api/work-orders` — Search

```
GET /api/work-orders?operatorCode=WIK&status=PENDING,ASSIGNED&kind=SUPPORT&techRegionCode=NRB-KAREN&limit=50 HTTP/1.1
```

Supports filters: `operatorCode`, `status` (comma-separated), `kind`, `jobTypeCode`, `assignedContractorId`, `assignedTechnicianId`, `techRegionCode`, `customerId`, `homepassId`, `originatingContextType`, `originatingContextRef`, `createdAfter`, `createdBefore`, `scheduledAfter`, `scheduledBefore`. Pagination via `limit` (max 100) + `cursor`. Returns header `Total-Count` for dispatcher pending-count widget.

Response 200:

```
{
  "items": [/* WO summaries */],
  "nextCursor": "eyJjcmVhdGVkQXQ...",
  "totalCount": 661
}
```

9. v1.0 stubs

The WO module is itself a v1.0 stub-retirement for `install_wo_stub` referenced by FUL-02 FRAMEWORK §6.2. Once this DD ships and is implemented, FUL-02 STEP-INSTALL switches from writing to `install_wo_stub` to calling `POST /api/work-orders` on this module.

The WO module has limited internal stubs:

9.1 `staff_notification_stub`

Same stub as FUL-02 FRAMEWORK §6.3. Used by `NotificationHandlerResource` when a Camunda user task requires dispatcher / contractor / NOC action and the staff-notification channel must fire (email, Slack, Teams). Retired when ICN-01 (Internal Communications module) ships.

9.2 `customer_stub`

Same stub as FUL-02 FRAMEWORK §6.1. Used for `GET /api/customers/{id}` reads when a WO needs customer name + contact info (rendered into print PDF, surfaced in BO UI). Retired when ILM-CFG-01 (Customer Service module) ships.

9.3 `wo_equipment_ref_stub`

Why this exists. When a Wananchi field tech finishes an installation or a support call where a device was replaced, the contractor records what was physically installed or swapped — brand, model, serial number, MAC address, ONU-ID. Workshop sources reference this capture happening at finalization time. SOPHIX has to retain this for four reasons:

1. **Contractor invoicing.** Finance settles with each contractor per visit; the visit is materially identified by what they installed. The `WorkOrderFinalized` event must carry equipment bindings so Finance can match contractor claims against actual installations.
2. **Future Inventory reconciliation.** When a real Inventory / Equipment Catalog module exists, stock movements (modem out of contractor warehouse → into customer premise) reconcile against WO equipment bindings. Without this data in v1.0 we'd have a discontinuity when the inventory module ships.
3. **Warranty + RPT.** Workshop: 6-month installation warranty, 3-month support warranty. If a customer reports an issue against a known-installed device (RPT WO), the warranty-eligibility check needs to look up "what device was installed when, by whom" — that lookup goes through the equipment-binding record on the prior WO.
4. **Cross-system consistency.** BroadHub (BH) and the provisioning layer hold their own view of what's at a customer premise; reconciling against SOPHIX's WO equipment bindings flags drift.

Why a stub. v1.0 has no Equipment Catalog / Inventory module. We can't reference equipment by FK to a non-existent module. The stub is the minimum data the WO module needs to record at finalization time so consumers (Finance event consumers in v1.0; future Inventory module later) can act. When the real Equipment Catalog module ships, this stub is retired and replaced with FK references to the proper catalog; the `WorkOrderFinalized` event payload shape is preserved — only the source-of-truth changes.

How it's used. Equipment bindings are written as one or more rows in `wo_equipment_ref_stub` AT WO finalization time, then summarised into the `equipmentBindings[]` array on the `WorkOrderFinalized` event. Pre-finalization, equipment intent (what the dispatcher told the field tech to bring) can be captured as a structured note (`note_kind = equipment_intent`). At finalization, the actual delivered equipment is recorded by the field tech / contractor dispatcher via a separate `POST /api/work-orders/{id}/equipment-bindings` endpoint (or as a structured note with `note_kind = equipment_installed`, depending on how flow satellites choose to capture it — framework supports both shapes).

```
CREATE TABLE wo_equipment_ref_stub (
  equipment_ref_id    TEXT PRIMARY KEY,           -- 'eqref_01HZX9...' ULID
  wo_id               TEXT NOT NULL,              -- FK to work_order
  operator_code       TEXT NOT NULL,              -- 'WIK'
  equipment_kind      TEXT NOT NULL,              -- 'HFC_MODEM', 'GPON_ONT', 'ATA', 'TV_DECODER', 'CABLE_DROP', 'SPLICE_
  brand               TEXT NULL,                  -- 'Cisco', 'Hitron', 'Huawei'
  model               TEXT NULL,                  -- 'EPC3925', 'CGNV4', 'HG8245H'
  serial_number       TEXT NULL,                  -- 'HFC-MODEM-XYZ123'
  mac_address         TEXT NULL,                  -- '00:1A:2B:3C:4D:5E'
  onu_id              TEXT NULL,                  -- '12' (GPON-specific)
  action              TEXT NOT NULL,              -- 'INSTALLED', 'REPLACED', 'RECOVERED', 'REUSED'
  replaced_serial     TEXT NULL,                  -- non-null when action='REPLACED'
  notes               TEXT NULL,                  -- free text
  recorded_by         TEXT NOT NULL,              -- BO user or worker ID
  recorded_at         TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_eqref_wo  ON wo_equipment_ref_stub (wo_id);
CREATE INDEX idx_eqref_op_kind ON wo_equipment_ref_stub (operator_code, equipment_kind);
CREATE INDEX idx_eqref_serial  ON wo_equipment_ref_stub (serial_number) WHERE serial_number IS NOT NULL;
```

Sample data:

equipment_ref_id	wo_id	operator	equipment_kind	brand	model	serial_number	mac_address	action
`eqref_01HZ_A1`	`wo_01HZ...A`	`WIK`	`HFC_MODEM`	`Hitron`	`CGNV4`	`HFC-MODEM-XYZ123`	`00:1A:2B:3C:4D:5E`	`INSTALLED`
`eqref_01HZ_A2`	`wo_01HZ...A`	`WIK`	`CABLE_DROP`	NULL	NULL	NULL	NULL	`INSTALLED`
`eqref_01HZ_B1`	`wo_01HZ...B`	`WIK`	`GPON_ONT`	`Huawei`	`HG8245H`	`GPON-ONT-ABC789`	`00:4F:AA:1B:2C:3D`	`REPLACED` (replaced_serial: `GPON-ONT-OLD-456`)
`eqref_01HZ_C1`	`wo_01HZ...C`	`WIK`	`ATA`	`Grandstream`	`HT801`	`ATA-LMN-321`	NULL	`INSTALLED`

Retirement path. When the future Equipment Catalog / Inventory module ships: (1) Equipment SKUs become first-class with their own DDL; (2) `wo_equipment_ref_stub.equipment_kind` becomes an FK to the new catalog; (3) `brand` / `model` columns collapse into the SKU reference; (4) The new module owns stock tracking, ownership-transfer events, return-to-warehouse flows. The WO module retains only the `(wo_id, equipment_ref)` linkage. Historical stub rows are migrated forward by mapping `(brand, model)` tuples to SKUs.

10. Framework rules

Cross-cutting rules every flow obeys. Naming pattern: `R-WO-FW-*`.

Rule ID	Statement
`R-WO-FW-1`	A WO is uniquely identified by `wo_id` (ULID). The `(operator_code, originating_context_type, idempotency_key)` triple is also UNIQUE to prevent duplicate creation by a retrying consumer.
`R-WO-FW-2`	`kind` ∈ {`INSTALLATION`, `SUPPORT`, `SHIFTING`} in v1.0. Future kinds added via `wo_flow_config` DDL row, not schema change.
`R-WO-FW-3`	`job_type_code` MUST exist in `wo_job_type_catalog` for the `(operator_code)` at creation time. Catalog enables/disables via `active` flag, not deletion.
`R-WO-FW-4`	`status` transitions follow the table in §3. Illegal transitions return 400 `ILLEGAL_STATE_TRANSITION` with current + attempted state.
`R-WO-FW-5`	`current_phase` is read-only outside `/transition-phase` API. Flows that don't use phases leave it NULL.
`R-WO-FW-6`	Every `status` change emits exactly one Kafka event (§7) and writes exactly one `wo_status_history` row.

Rule ID	Statement
R-WO-FW-7	Reassignment (/reassign) is allowed in any non-terminal status (PENDING, ASSIGNED, IN_PROGRESS, FINALIZATION_PENDING). Status does not change; wo_assignment_history row written; WorkOrderReassigned emitted.
R-WO-FW-8	assigned_technician_id is set ONLY by /assign with technicianId (TECHNICIAN mode), /claim with claimedByExternalDispatcher=false (TEAM mode internal-tech path), or /reassign. The external-dispatcher path (/claim with claimedByExternalDispatcher=true) leaves assigned_technician_id NULL and sets claimed_by_dispatcher instead.
R-WO-FW-9	A technician without the skill for the WO's job_type_code (per wo_technician_skill) MUST NOT be assignable / claimable in any internal-team flow. Server-side enforcement on /assign (TECHNICIAN mode), /reassign, /claim (internal-tech path). Returns 400 TECHNICIAN_SKILL_MISMATCH. Skill validation is NOT performed for external-team TEAM-mode flows; the contractor's dispatcher is responsible for picking a skilled person.
R-WO-FW-10	A contractor whose tech_regions_served does not include the WO's tech_region_code MUST NOT be assignable. Returns 400 CONTRACTOR_REGION_MISMATCH. Special value "ALL" in tech_regions_served bypasses this check (used for internal NOC).
R-WO-FW-11	Schedule resolution at assignment/claim time follows §6.5: for TECHNICIAN mode, the technician's own wo_schedule row (scope=TECHNICIAN) overrides the team's row (scope=TEAM); if no row at either scope for that day-of-week → not available. For TEAM mode at assignment time, the team's wo_schedule row alone is consulted. For internal-tech claim, per-tech override applies. The scheduled_at hour must fall within [start_hour, end_hour). Returns 400 TECHNICIAN_NOT_AVAILABLE (TECHNICIAN mode / internal-tech claim) or 400 TEAM_NOT_AVAILABLE (TEAM-mode assignment).
R-WO-FW-11a	Assignment mode is TECHNICIAN when /assign (or /reassign) is called with a non-null technicianId; TEAM when called with technicianId=null. Auto-assignment Drools may choose either mode by returning a ContractorAssignmentDecision with or without technicianId. TECHNICIAN mode requires a matching wo_technician row to exist; for external contractors that don't register technicians, only TEAM mode is usable in practice.
R-WO-FW-11b	In TEAM mode the WO remains in status ASSIGNED with claimed_at = NULL until /claim is called via one of two paths: (1) an internal technician claims by supplying their technicianId; or (2) an external contractor's dispatcher acknowledges by supplying claimedByExternalDispatcher=true + claimedBy=<BO user>. The /start API returns 400 WORK_ORDER_NOT_CLAIMED until claim happens.
R-WO-FW-11c	Claim is atomic: UPDATE work_order SET <claim_columns> WHERE wo_id = ? AND claimed_at IS NULL. Internal-tech path sets assigned_technician_id + claimed_at. External-dispatcher path sets claimed_by_dispatcher + claimed_at. Second claim returns 409 ALREADY_CLAIMED with the winning party recorded.
R-WO-FW-11d	After a successful TEAM-mode claim (either path), framework emits the matching event (WorkOrderClaimedByTechnician or WorkOrderClaimedByDispatcher) AND signals the notification module to cancel pending claim-prompt notifications for the rest of the audience (team members for internal path; other dispatchers for external path if applicable).
R-WO-FW-11e	A technician already owning a non-terminal WO MUST NOT be assignable to another. A WO is "owned" by a tech when assigned_technician_id = X AND status IN (ASSIGNED, IN_PROGRESS, FINALIZATION_PENDING). Server-side enforcement on /assign (TECHNICIAN mode), /reassign (when supplying a new tech), and /claim (internal-tech path). Returns 400 TECHNICIAN_HAS_PENDING_WO with the existing woId in the error body. The check uses SELECT wo_id FROM work_order WHERE assigned_technician_id = $1 AND status IN ('ASSIGNED','IN_PROGRESS','FINALIZATION_PENDING') LIMIT 1. Being a *candidate* on TEAM-mode WOs (i.e., on the notification list awaiting claim) does NOT count as ownership; only a successful claim does. The external-dispatcher path does not perform this check (no individual tech tracked). When /reassign moves a WO from tech A to tech B, A is freed in the same transaction that locks B.
R-WO-FW-12	Notes are append-only by default. Only note kinds with append_only = false in wo_note_kind_registry can supersede earlier rows (via superseded_by_note_id).

Rule ID	Statement
R-WO-FW-13	Note JSONB payload MUST validate against wo_note_kind_registry.schema_jsonb when present. Returns 400 NOTE_SCHEMA_VIOLATION.
R-WO-FW-14	Attachments are append-only. No delete operation (mistaken uploads handled by adding a corrective attachment + dispatcher_note).
R-WO-FW-15	Finalization is 2-step: first-confirm → FINALIZATION_PENDING; second-confirm → COMPLETED (after checklist passes).
R-WO-FW-16	Finalization checklist read from wo_finalization_requirements matching (operator_code, kind, job_type_code). Specific-job-type row overrides job_type_code=NULL default row.
R-WO-FW-17	When wo_final_reason_catalog.triggers_escalation_event = true, the framework emits BOTH WorkOrderFinalized AND WorkOrderEscalationCandidate. The escalation candidate event carries the final reason and last findings note.
R-WO-FW-18	Cancellation is allowed from any non-terminal status. Cancellation message interrupts any active Camunda subprocess via the framework-provided boundary event (§2.4).
R-WO-FW-19	All *_at timestamp columns are set exactly once on first arrival in the corresponding status. The framework worker wo-emit-state-event guarantees this.
R-WO-FW-20	The master_wo_id + link_type pair: either both NULL or both set. v1.0 link types: ESCALATION, REPEAT_WITHIN_WARRANTY, PAIRED. Future link types added without schema change.
R-WO-FW-21	Concurrent WOs against the same customer_id are allowed; no framework-level constraint. Business rule (e.g., "no two open installs for the same homepass") is consumer's responsibility.
R-WO-FW-22	The Camunda processInstanceId on work_order is set after instance start; rows with NULL process_instance_id are transient (within seconds) and indicate an in-flight create. Background worker reconciles abandoned creates: drops rows older than 5 minutes with no instance ID.
R-WO-FW-23	The audit log (wo_audit_log) captures every state change, every reassignment, every note append, every attachment add, every print, every cancel, every finalize-revert. Used for SOX-style audit and for the dispatcher "WO history" view.
R-WO-FW-24	All consumer-facing REST endpoints accept idempotency via the Idempotency-Key HTTP header (separate from the idempotencyKey body field used at creation). Retries within 24h return cached response.
R-WO-FW-25	Equipment bindings captured against a WO are recorded in wo_equipment_ref_stub and summarised into the equipmentBindings[] array of the WorkOrderFinalized event. Bindings are flow-driven: flow satellites decide when bindings are required (e.g., Installation flow requires ≥1 binding at finalization; Support flow allows zero or many; Shifting flow allows recovered bindings at GSD and new bindings at GSR). The framework provides the storage + event-payload mechanics; the flow defines the requirement via wo_finalization_requirements.required_note_kinds (when bindings are captured as notes) or via a flow-specific check.

11. KE workshop coverage — what we ship vs what we don't

Per memory rule #13: three labels — what KE workshops say the business needs, what the TZ codebase covers today, and the gap + V3 design rationale.

11.1 KE workflow target (per workshop 2026-04-29 + transcript Bildad Gitonga 2026-04-30)

The Wananchi KE business runs every commercial + technical operation through a Work Order. The framework must support:

- WO creation for Installation, Support, GPON support, HFC support, GSD (Shifting Disconnect), GSR (Shifting Reconnect) flows.
- The KE Job Type enum (~30 codes spanning Address Management, Service Changes, Installations, Support/Service Calls, Equipment, Disconnections/Relocations, Network Migrations, Quality Control, Contractor Invoicing, Retention, Other/Specialised — full list in wo_job_type_catalog seed).

- The KE Sub Status enum (ACT, ADD, CMP, OFC, SED, STF, TST, VIP, INA) — captured as a side-effect of WO finalization, consumed by ILM-CFG-01 (when it ships).
- Job-Type-driven dropdowns (Initial Reason picklist, New Sub Status picklist filter per Job Type).
- Contractor + team + skills model — Service Plan Type (INT/OTH/PHN/VID) + Technician Work Type (per Job Type code).
- WO assignment via Group → pick → auto-assign pattern (acknowledged SLA pain; redesign deferred to Phase 2 per GAP).
- Easydocsis optical check before finalization (mandatory for GPON installs and GPON support).
- Structured optical readings captured in WO notes (OLT PORT, ONU-ID, DISTANCE, OPTICAL TEMP, ONT RX/TX POWER, OLT RX POWER, IP, LASTOWNCAUSE, BATT).
- HFC signal readings captured in WO notes (downstream level, upstream level, SNR).
- WO Notes for handler coordination — structured kinds (findings, solution, customer_appointment) + free-text.
- Image attachments with category + description.
- Required-before-closing checklist: findings, solution, signal/optical stats, final reason, photos.
- 2-step finalization confirm: first confirm saves, second confirm closes.
- WO Escalation pattern A (Support): finalize original + raise QCS WO to NOC/maintenance (separately trackable).
- WO Escalation pattern B (Installation): same install WO is reassigned (not closed) to maintenance/construction; comes back to install contractor for finalization once unblocked.
- Shifting WO flow (GSD + GSR) — sequential, finalization-order rule (disconnect first), free of charge.
- Warranty model: 3 months Support, 6 months Installation. Repeat-within-warranty → RPT linkage.
- SLA timestamps captured per WO (received_at, finalized_at, intermediates). Calc external for v1.0.
- WO print to PDF (one at a time).
- Audit log of every change.

11.2 TZ codebase coverage today

To be filled in after codebase review. No claims about what the TZ-deployed Sophix codebase does are recorded here until the codebase has been read directly.

The cartography phase (per the pending plan) will produce this content. Once the codebase is available, this section will be filled with verified statements about which WO-framework capabilities exist and which do not — replacing this placeholder.

11.3 V3 design rationale

This satellite's V3 design choices follow from the KE workshop requirements (§11.1) and the cross-operator deployment-model variation discussed throughout this DD:

1. **Sub Status semantics are Wananchi-specific.** "INA on GSD finalization, STF on GSR finalization" is a coupling between WO and account-status. V3 captures it as a finalization side-effect emitted on Kafka (`WorkOrderFinalized` event payload), consumed by ILM-CFG-01.
2. **The structured-notes mechanism.** V3 has typed, schema-validated notes registered per operator. This is the foundation for Easydocsis-readings, customer-appointment-tracking, all future field-tech-feedback channels — without forking schema per operator.
3. **The required-before-finalize checklist.** V3 enforces via `wo_finalization_requirements`. Configurable per operator + per job type without code change.
4. **Multi-phase BPMN.** KE workshop confirms single-WO-with-2-phases for Shifting. V3 framework supports both patterns (single phase + multi-phase) so all operators can compose without schema fork.
5. **Master/sub linkage primitive.** Sekou's stated SOPHIX design ask. V3 ships the primitive; v1.0 flows don't use it; Ticketing module (Phase B) drives Pattern A escalation through it.
6. **Per-operator BPMN composition.** V3 reserves operator-suffixed BPMN keys (`WorkOrder_Installation_WIK`, future `_WUG`, `_WTZ`, `_YASSN`) — same code module, different orchestration per operator's regulatory + business reality. v1.0 ships WIK seed; later operators add config rows, not code.

7. **Native SLA timestamp capture (Phase 1) sets up future SLA engine (Phase 2).** V3 captures timestamps inside the WO row at known transition points so a future FUL-OPS-SLA module can subscribe and compute without external tooling.
8. **What V3 does NOT bring (per GAP roadmap):** Native SLA engine (Phase 2), Tech Route view, formal Customer Appointment field, group-assign redesign, Google Maps integration, Inventory/contractor-stocks, Tech Performance dashboard, full reporting, mobile field app, native field-team feedback channel (WhatsApp stays). These are explicitly Phase 2.

Field Audit feature is NOT used in Wananchi (workshop confirmation) — V3 drops it entirely.

Mapping to TZ baseline (§11.2) — what V3 adds vs what the codebase already supports — will be filled in after codebase review.

12. Cross-DD coordination

12.1 Module owner

The WO module is owned by the Fulfillment & Operations team. The framework satellite documented here is the foundation that all WO flow satellites (Installation, Support, Shifting in Phase B) build on.

12.2 Impacted modules

Module	Direction	Contract
FUL-02 (Order Capture)	FUL-02 → WO	FUL-02 STEP-INSTALL calls `POST /api/work-orders` with `kind=INSTALLATION`, `originatingContextType=ORDER`. WO emits `WorkOrderFinalized` / `WorkOrderCancelled`; FUL-02 consumes both. Retires `install_wo_stub` in FUL-02 FRAMEWORK §6.2 once WO module ships.
Future Ticketing module	Ticketing → WO	Ticketing creates Support WOs with `kind=SUPPORT`, `originatingContextType=TICKET`. Subscribes to `WorkOrderEscalationCandidate` to decide whether to spawn escalation WOs (Pattern A).
Future OSR-* modules	OSR-* → WO	OSR-MOVE creates Shifting WOs; OSR-ADD/CHANGE/DISCONNECT create their own WO kinds (added to `wo_flow_config` as future kinds).
RLM-CFG-01 (HomePass)	WO ← RLM	WO reads `homepass.tech_region_code` for contractor routing. WO does not write to HomePass. The Shifting RECONNECT phase emits an event consumed by RLM-CFG-01 to update the customer's homepass binding.
ILM-CFG-01 (Customer Service — future)	WO ← ILM-CFG-01 (read); WO → ILM-CFG-01 (event)	WO reads customer profile for print PDF + BO UI display. WO emits `WorkOrderFinalized` carrying Sub Status side-effect payload (e.g., `accountSubStatusChange: {to: INA, fromKind: GSD}`) which ILM-CFG-01 consumes to update account.sub_status. v1.0 uses `customer_stub` (§9.2).
ICN-01 (Internal Communications — future)	WO → ICN-01	NotificationHandlerResource dispatches user-task notifications to dispatchers, contractors, NOC. v1.0 uses `staff_notification_stub` (§9.1).
FOUNDATION_CAMUNDA	WO depends on	Per memory rule #12, all Camunda concept terminology in this DD inlined.
FOUNDATION_DROOLS	WO depends on	Auto-assignment Drools KJAR per operator.
PLM-CFG-01 / SIP-01	indirect	WO references `customer_id` / `homepass_id`; the underlying Service / Package model lives in PLM-CFG-01 + SIP-01. WO doesn't read those directly.
BIL-* (Billing modules)	WO → Finance	`WorkOrderFinalized` consumed by future Finance/Billing for contractor invoicing (workshop: "visit is tracked + contractor can be invoiced"). Out of v1.0 WO scope.

12.3 Boundary clarifications

What is in scope for this framework satellite and what is not:

- ■ WO state machine, REST API, schemas, events, Camunda integration, configuration tables, contractor/team/skills model, audit log, framework rules.
- ■ Installation flow BPMN — DD_WO-01-FLOW-INSTALLATION (Phase B).
- ■ Support flow BPMN — DD_WO-01-FLOW-SUPPORT (Phase B).
- ■ Shifting flow BPMN — DD_WO-01-FLOW-SHIFTING (Phase B).
- ■ QCS escalation policy decision — owned by future Ticketing module DD.
- ■ Pattern A escalation new-WO creation — owned by Ticketing.
- ■ Native SLA engine — owned by future FUL-OPS-SLA DD (Phase 2).
- ■ Mobile field tech app — owned by future Field Tech App DD (Phase 2, conditional).
- ■ Google Maps integration — owned by BO UI dispatcher tool (Phase 2).
- ■ Tech Performance dashboard — owned by future Reporting module (Phase 2).
- ■ Inventory / contractor stocks — owned by future Inventory module.
- ■ Field Audit feature — explicitly NOT used per workshop (drop entirely).

12.4 Dependencies (build order)

1. RLM-CFG-01 (HomePass) — already designed; ships before WO.
2. FUL-02 FRAMEWORK — already designed; `install_wo_stub` retires when this WO framework ships.
3. **DD_WO-01-FRAMEWORK-v1.0 (this DD)** — Phase A foundation.
4. DD_WO-01-FLOW-INSTALLATION-v1.0 (Phase B).
5. DD_WO-01-FLOW-SUPPORT-v1.0 (Phase B).
6. DD_WO-01-FLOW-SHIFTING-v1.0 (Phase B).
7. ILM-CFG-01 (Customer Service) + ICN-01 (Internal Communications) — retire `customer_stub` + `staff_notification_stub` respectively.
8. Future Ticketing module — drives Pattern A QCS escalation through master/sub linkage primitive.
9. Future Inventory module — retires `equipment_ref_stub`.
10. Future FUL-OPS-SLA — consumes WO SLA timestamps.

12.5 Migration approach

When this framework + its Phase B flow satellites ship:

1. **DDL deployment** — create all `wo_*` tables; seed `wo_flow_config`, `wo_job_type_catalog`, `wo_note_kind_registry`, `wo_finalization_requirements`, `wo_final_reason_catalog`, `wo_contractor`, `wo_team`, `wo_technician`, `wo_technician_skill`, `wo_schedule` for WIK.
2. **BPMN deployment** — deploy `WorkOrder_Installation_WIK.bpmn`, `WorkOrder_Support_WIK.bpmn`, `WorkOrder_Shifting_WIK.bpmn` to Camunda. Deploy Drools KJARs.
3. **FUL-02 cutover** — flip FUL-02 STEP-INSTALL configuration from `useStub=true` to `useStub=false`. STEP-INSTALL starts calling `POST /api/work-orders` instead of writing to `install_wo_stub`. Stub table dropped after 30-day grace period.
4. **WUG / WTZ / YASSN onboarding** — when each new operator's flows are designed, add operator-suffixed BPMN keys (`_WUG`, `_WTZ`, `_YASSN`), add corresponding `wo_flow_config` + `wo_job_type_catalog` + `wo_note_kind_registry` + `wo_finalization_requirements` + `wo_final_reason_catalog` rows. No code change in this framework.

13. Runtime walkthrough — narrative

The framework reference manual above tells you the surface. The walkthrough below tells you how a real WO moves through the system end-to-end. Concrete steps, named workers, sample payloads. Two scenarios: a happy-path HFC installation and

a Shifting flow showing the multi-phase mechanism.

13.1 Happy path — HFC installation WO

Scenario. Customer Jane Mwangi has just paid for her new HFC installation. FUL-02 STEP-INSTALL just got `OrderPaymentReceived`; it now creates the install WO.

Step 1 — FUL-02 STEP-INSTALL calls `POST /api/work-orders`.

FUL-02's external task worker `order-create-install-wo` (lives in FUL-02 module) calls:

```
POST /api/work-orders HTTP/1.1
Host: work-order.sophix.internal
Authorization: Bearer <ful-02-svc-creds>

{
  "operatorCode":           "WIK",
  "kind":                   "INSTALLATION",
  "jobTypeCode":            "H01",
  "customerId":             "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "homepassId":             "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "originatingContextType": "ORDER",
  "originatingContextRef":  "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "idempotencyKey":         "ful-02-step-install-ord_01HZ...A",
  "metadata":               {"customerName":"Jane Mwangi","contactNumber":"+254712345678"}
}
```

The framework: inserts `work_order` row with `status=PENDING`; calls Camunda `POST /engine-rest/process-definition/key/WorkOrder_Installation_WIK/start` with `businessKey=wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2`; updates the row with `process_instance_id`; emits `WorkOrderCreated` to `sophix.work-orders.created`. Returns 201 with the new `woId`. FUL-02's worker stores `woId` on the order and completes its Camunda task.

Step 2 — Camunda's `WorkOrder_Installation_WIK` BPMN starts. First external task is `wo-validate-create`. The framework worker subscribes to that topic, picks the task, validates basic invariants (operator exists, homepass tech_region resolved, customer reachable). Completes.

Step 3 — BPMN's next step is `wo-auto-assign-contractor` external task (auto-assignment path). Framework worker reads `wo_flow_config.drools_kjar_ref` (`wo-installation-wik-v1.0.kjar`), invokes Drools with the WO context. Drools picks `ctr_Z071H` (covers `NRB-KAREN` where the HomePass lives), `team_rapid_response`, leaves `technicianId` NULL (contractor's own dispatcher picks the tech). The worker:

- Calls internal service to update `work_order` row: `status=ASSIGNED`, `assigned_contractor_id=ctr_Z071H`, `assigned_team_id=team_rapid_response`, `assigned_at=NOW()`.
- Writes `wo_status_history` row: PENDING → ASSIGNED, triggered_by=`auto_assign_worker`.
- Writes `wo_assignment_history` row: initial_assignment.
- Emits `WorkOrderAssigned` to `sophix.work-orders.assigned`.

Completes the Camunda task.

Step 4 — Camunda BPMN parks at a user task `await-contractor-dispatcher-pick`. The user task fires an HTTP connector → `NotificationHandlerResource` → `staff_notification_stub` → email to contractor Z071H's dispatcher distribution list: "New install WO awaiting tech assignment: wo_01HZ...Z2 for Jane Mwangi at NRB-KAREN."

Step 5 — Contractor dispatcher picks a tech via BO UI. UI calls `POST /api/work-orders/wo_01HZ...Z2/assign`:

```
{
  "contractorId":     "ctr_Z071H",
  "teamId":           "team_rapid_response",
  "technicianId":     "tech_01HZ_KAREN_03",
  "scheduledAt":      "2026-05-18T14:00:00Z",
  "assignmentReason": "dispatcher_pick"
}
```

Framework validates skills (tech_01HZ_KAREN_03 has skill `INT + H01`), region match, schedule (Mon 18-May is available). Passes. Updates row: `assigned_technician_id`, `scheduled_at`, `dispatched_at = NOW()`. Sends Camunda message `WorkOrderUserActionReceived` correlating on `businessKey=wo_01HZ...Z2` and message variable `actionKind=assignment_complete`. Camunda BPMN wakes the user task and proceeds. Emits `WorkOrderDispatched`.

Step 6 — Camunda BPMN parks at a user task `await-tech-start` (contractor's dispatcher prints WO, tech goes to site). The tech arrives, contractor dispatcher confirms via `POST /api/work-orders/wo_01HZ...Z2/start`. Status →

IN_PROGRESS. `in_progress_at` set. Emits `WorkOrderInProgress`.

Step 7 — On-site work. Tech captures notes + attachments. Tech adds notes via the BO UI (or future mobile app, or WhatsApp pasted into BO UI by contractor):

- `POST .../notes`: `noteKind=findings`, `contentText="Modem installed at living-room wall outlet. Drop cable run from utility pole to wall plate completed. CMTS sees modem online."`
- `POST .../notes`: `noteKind=solution`, `contentText="Standard installation completed without issues."`
- `POST .../notes`: `noteKind=hfc_signal_readings`, `contentJsonb={"downstreamLevel":-3.2,"upstreamLevel":42.1,"snr":38.5,"package":"INET_50M"}`.
- `POST .../attachments`: `category=setup_photo`, file `modem_wall_mount.jpg`.
- `POST .../attachments`: `category=speed_test`, file `ookla_50_10_mbps.png`.

Each call writes the corresponding row, emits `WorkOrderNoteAppended` or `WorkOrderAttachmentAdded`, writes `wo_audit_log` row.

Step 8 — Contractor dispatcher first-confirms finalize via BO UI. UI calls `POST .../finalize-first-confirm`:

```
{
  "finalReasonCode": "INSTALL_COMPLETED_CUSTOMER_UP",
  "confirmedBy":     "ctr_Z071H_dispatcher_01"
}
```

Framework: verifies status=IN_PROGRESS; writes `final_reason_set` note row with the chosen reason; sets `final_reason_code` on `work_order`; status → FINALIZATION_PENDING; `finalization_pending_at = NOW()`; emits `WorkOrderFinalizationPending`. Sends Camunda message `WorkOrderUserActionReceived` with `actionKind=finalize_first_confirm`. Camunda BPMN advances to next user task `await-finalize-second-confirm`.

Step 9 — Contractor dispatcher second-confirms via BO UI. UI calls `POST .../finalize-second-confirm` (no body). Framework's `wo-enforce-finalize-checklist` worker runs:

- Read `wo_finalization_requirements` for `(WIK, INSTALLATION, H01)`: `required_note_kinds=["findings","solution","hfc_signal_readings","final_reason_set"]`, `required_attachment_categories=["setup_photo","speed_test"]`.
- Count: findings=1 ✓, solution=1 ✓, hfc_signal_readings=1 ✓, final_reason_set=1 ✓. Attachments: setup_photo=1 ✓, speed_test=1 ✓. All present.
- Pass. Update row: status=COMPLETED, `completed_at = NOW()`. Emit `WorkOrderFinalized` (see §7 sample). Note `triggers_escalation_event=false` for this final reason → no escalation candidate event. Camunda BPMN's end event reached; process instance closes.

Step 10 — FUL-02 STEP-INSTALL consumes `WorkOrderFinalized`. FUL-02's Kafka consumer routes the event to STEP-INSTALL's `wait-install-wo-finalized` message catch, correlating on `originatingContextRef=ord_01HZ...A`. FUL-02 BPMN advances; activation triggers.

Total elapsed time in this scenario: ~6 hours (creation at 09:00, finalize at ~14:50).

13.2 Team-level assignment with race-to-claim — internal team

Scenario. A Support WO comes in for a GPON internet outage at NRB-KAREN. The dispatcher decides not to pre-assign a specific technician — she assigns the WO to the in-house team and lets whoever is free claim it.

Step 1 — Dispatcher assigns to TEAM. UI calls:

```
POST /api/work-orders/wo_01HZ...T2/assign HTTP/1.1

{
  "contractorId":    "ctr_Z071H",
  "teamId":          "team_rapid_response",
  "technicianId":    null,
  "scheduledAt":     "2026-05-18T14:00:00Z",
  "assignmentReason":"manual_team_assignment"
}
```

Framework validates that `ctr_Z071H` covers `NRB-KAREN`; that the team's `wo_schedule` row for Monday includes 14:00 (it does — Mon 09–18). Updates row: `status = ASSIGNED`, `assignment_mode = TEAM`, `assigned_contractor_id = ctr_Z071H`, `assigned_team_id = team_rapid_response`, `assigned_technician_id = NULL`, `claimed_at = NULL`, `claimed_by_dispatcher = NULL`, `assigned_at = NOW()`. Emits `WorkOrderAssigned`.

Note: skill validation is not performed at team-mode assignment time. Each technician's individual skill match is checked at claim time.

Step 2 — Notification fans out. `staff_notification_stub` (v1.0) sends a push/email to every active tech on `team_rapid_response` — say there are 3: `tech_01HZ_KAREN_03`, `tech_01HZ_KAREN_05`, `tech_01HZ_KAREN_07`. Message: "WO wo_01HZ...T2 awaiting claim for GPON outage, scheduled Mon 14:00. Tap to claim."

Step 3 — Race. `tech_01HZ_KAREN_05` and `tech_01HZ_KAREN_07` both tap claim at nearly the same moment. Both BO UIs / mobile apps call:

```
POST /api/work-orders/wo_01HZ...T2/claim HTTP/1.1
(tech_01HZ_KAREN_05)
{
  "claimedByExternalDispatcher": false,
  "technicianId":                "tech_01HZ_KAREN_05",
  "claimedAt":                   "2026-05-18T13:42:00.123Z"
}

POST /api/work-orders/wo_01HZ...T2/claim HTTP/1.1
(tech_01HZ_KAREN_07)
{
  "claimedByExternalDispatcher": false,
  "technicianId":                "tech_01HZ_KAREN_07",
  "claimedAt":                   "2026-05-18T13:42:00.198Z"
}
```

Framework runs both through the atomic update:

```
UPDATE work_order
   SET assigned_technician_id = $1, claimed_at = NOW()
 WHERE wo_id = 'wo_01HZ...T2'
   AND claimed_at IS NULL;
```

But first, skill + schedule validation runs for each tech. Both have skill `INT + GP3`; both have Mon 14:00 within their resolved schedule windows. Both pass validation. The first call (tech_05) wins the atomic update: 1 row affected. The second call (tech_07) gets 0 rows affected → framework returns 409:

```
{
  "error": "ALREADY_CLAIMED",
  "claimedBy": "tech_01HZ_KAREN_05",
  "claimedAt": "2026-05-18T13:42:00.123Z"
}
```

Step 4 — Winner takes the WO. For tech_05's successful claim: framework emits `WorkOrderClaimedByTechnician` and signals the notification module to cancel pending notifications for tech_03 and tech_07. BO UI on tech_03 and tech_07 devices updates: the WO disappears from their "available to claim" list. tech_05's BO UI now shows the WO in their "my active WOs" list.

Step 5 — Tech proceeds. tech_05 drives to site, calls `POST /start`, captures notes + attachments + readings, two-step finalizes. From step 6 onward the flow is identical to §13.1 — TECHNICIAN mode and post-claim TEAM-mode-internal behave identically once `assigned_technician_id` is set.

13.3 Team-level assignment — external team

Scenario. Same kind of Support WO but in NRB-EASTLANDS, which is served by `ctr_ADRIAN` — an external contractor. ADRIAN works with rotating freelance technicians whom Wananchi doesn't track. There are no `wo_technician` rows for ADRIAN.

Step 1 — Dispatcher assigns to TEAM. UI calls:

```
POST /api/work-orders/wo_01HZ...T3/assign HTTP/1.1

{
  "contractorId":    "ctr_ADRIAN",
  "teamId":          "team_adrian_eastlands_install",
  "technicianId":    null,
  "scheduledAt":     "2026-05-18T14:00:00Z",
  "assignmentReason":"manual_team_assignment"
}
```

Framework validates region (`ctr_ADRIAN` covers `NRB-EASTLANDS`) and team schedule (Mon 14:00 in window). Updates row exactly as §13.2 Step 1 but with the external contractor's values. Emits `WorkOrderAssigned`.

Step 2 — Notification routed to dispatcher inbox. The contractor's dispatcher account (`dispatcher_user_adrian_01`) receives the alert via the channel the integration is set up for (email, Slack, WhatsApp). Wananchi-side support staff who coordinate with ADRIAN may also see the alert.

Step 3 — Dispatcher acknowledges receipt. `dispatcher_user_adrian_01` opens the WO in BO UI and clicks "Acknowledge for ADRIAN":

```
POST /api/work-orders/wo_01HZ...T3/claim HTTP/1.1

{
  "claimedByExternalDispatcher": true,
  "claimedBy":                   "dispatcher_user_adrian_01",
  "claimedAt":                   "2026-05-18T13:45:00Z"
}
```

Framework: verifies WO is in ASSIGNED + TEAM mode; verifies `dispatcher_user_adrian_01` is a registered BO user authorised to act for `ctr_ADRIAN`. Atomically updates `claimed_by_dispatcher = dispatcher_user_adrian_01`, `claimed_at = NOW()`. `assigned_technician_id` stays NULL. Emits `WorkOrderClaimedByDispatcher`.

Step 4 — Field work proceeds. ADRIAN's dispatcher sends one of their freelance technicians to the site via WhatsApp / their own internal channels. The freelancer fixes the issue. The dispatcher reads the freelancer's WhatsApp report and posts notes + attachments to the WO via BO UI — calls all attribute to `dispatcher_user_adrian_01` as the `appended_by`. The framework records who acted on the SOPHIX side; it never learns the freelancer's identity.

```
POST /api/work-orders/wo_01HZ...T3/start HTTP/1.1
{ "startedBy": "dispatcher_user_adrian_01" }
```

Status → IN_PROGRESS. Then notes + attachments + finalize follow normally, all attributed to the dispatcher user.

Step 5 — Finalize. Same 2-step confirm. `WorkOrderFinalized` event carries `assignedTechnicianId = null`, `claimedByDispatcher = dispatcher_user_adrian_01`. Consumers (Finance for contractor invoicing, NOC dashboards) handle the null `assignedTechnicianId` gracefully — contractor invoicing settles at the team / contractor level for external WOs.

13.4 Multi-phase — Shifting flow (GSD → GSR)

Scenario. Customer John Otieno is relocating from `hp_01HX...OLD` (NRB-KAREN) to a new address `hp_01HX...NEW` (NRB-WESTLANDS). Shifting is free. CSR raises the shifting WO via BO UI.

Step 1 — CSR creates the WO. UI calls `POST /api/work-orders`:

```
{
  "operatorCode":           "WIK",
  "kind":                   "SHIFTING",
  "jobTypeCode":            "GSD",
  "customerId":             "cust_01HY...",
  "homepassId":             "hp_01HX...OLD",
  "originatingContextType": "BACKOFFICE",
  "originatingContextRef":  "csr_user_42",
  "idempotencyKey":         "shift-csr_42-2026-05-18-09-30",
  "metadata":               {"newHomepassIdAtCreation":"hp_01HX...NEW","customerName":"John Otieno"}
}
```

Framework creates the WO with `status=PENDING`, starts `WorkOrder_Shifting_WIK.bpmn`. The Shifting flow's BPMN has a top-level structure: `DISCONNECT subprocess → phase-transition message catch → RECONNECT subprocess`. Camunda starts in DISCONNECT subprocess.

Step 2 — DISCONNECT phase runs. `current_phase` is set to `DISCONNECT` by the first external task. GSD has `requires_site_visit=false` (per `wo_job_type_catalog`); no tech assignment needed; auto-completes after a dispatcher_note is captured + a system action removes the old HomePass binding.

Framework: writes `dispatcher_note` to `wo_notes` ("Customer relocating to Plot 47, Westlands"); transitions status PENDING → IN_PROGRESS with `current_phase=DISCONNECT`; emits `WorkOrderPhaseTransitioned` with `{previousPhase:null,newPhase:"DISCONNECT"}`; emits `WorkOrderInProgress`.

The Shifting flow's DISCONNECT subprocess emits its own flow-specific event consumed by ILM-CFG-01: account sub_status → INA. (That's flow-owned, not framework-owned; documented in DD_WO-01-FLOW-SHIFTING.)

Step 3 — Phase transition DISCONNECT → RECONNECT. When the DISCONNECT subprocess completes, the Shifting BPMN sends an internal Camunda message that the framework's `/transition-phase` API converts to the next phase. Alternatively, the BPMN can directly transition without API call by setting `current_phase` via a service task.

Either way, the row updates: `current_phase = RECONNECT`. Status stays IN_PROGRESS. Emits `WorkOrderPhaseTransitioned` with `{previousPhase:"DISCONNECT",newPhase:"RECONNECT"}`.

Also, the WO's `job_type_code` updates from `GSD` → `GSR` (since the same WO row carries the active job_type). The flow's BPMN sets this via a service task. This is unusual (most WOs have one job_type for life) but legitimate for the Shifting multi-phase flow.

Step 4 — RECONNECT phase runs (full WO flow). With `job_type_code=GSR`, `requires_site_visit=true`. The flow's BPMN does auto-assignment for the new region (NRB-WESTLANDS): picks `ctr_GTECH`, `team_provision_westlands`. Status → ASSIGNED. Dispatcher picks `tech_01HZ_WESTLANDS_02`, schedules for 2026-05-19 10:00 — same `/assign` API as installation. Status → IN_PROGRESS again (well, stays IN_PROGRESS; the BPMN tracks ASSIGNED-within-RECONNECT internally). Tech arrives, captures `findings`, `solution`, `optical_readings`, `new_address`, photo. First-confirm + second-confirm as in §13.1.

Step 5 — RECONNECT finalization. `wo_finalization_requirements` row for `(WIK, SHIFTING, GSR)` requires `findings`, `solution`, `optical_readings`, `new_address`, `final_reason_set`. Checklist passes. Status → COMPLETED. Emits `WorkOrderFinalized` carrying the `new_address` metadata. ILM-CFG-01 consumes and updates the customer's home_pass binding to `hp_01HX...NEW`; sub_status → STF (per flow side-effect contract).

Throughout. Every state change, note, attachment, reassignment is logged in `wo_status_history`, `wo_assignment_history`, `wo_audit_log`. SLA timestamps captured at every transition. A future FUL-OPS-SLA module can compute "Shifting WO total duration", "Shifting WO disconnect-leg duration", "Shifting WO reconnect-leg duration" from these timestamps without changing this framework.

End of DD_WO-01-FRAMEWORK-v1.0.